

DOCUMENTO DE ARQUITECTURA Y AUDITORÍA TÉCNICA

Sistema de streaming end-to-end desplegado

Kinesis → Apache Flink → Apache Iceberg / Glue → Amazon Redshift

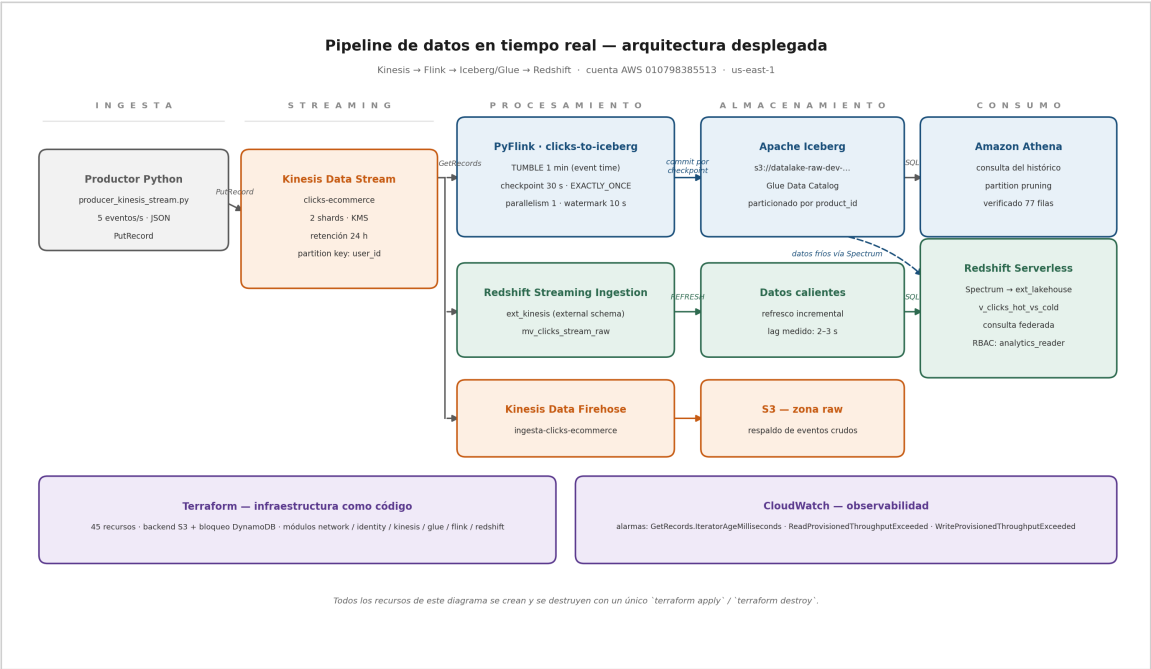


Figura 1. Arquitectura del pipeline desplegado: las cinco capas del flujo de datos y las dos capas transversales que las gobiernan.

Autor	Andrés Capo Plaza
Proyecto	Capstone — Pipeline de datos en tiempo real sobre AWS
Repositorio	github.com/AndresCapoPlaza/dataops-terraform-aws
Cuenta / región	010798385513 · us-east-1 (Norte de Virginia)
Fecha de las evidencias	10 de septiembre de 2026

1. Resumen ejecutivo

Este documento audita un pipeline de datos en tiempo real desplegado en AWS y operado de punta a punta: un evento de clic entra por Amazon Kinesis Data Streams, lo procesa Apache Flink con ventanas de un minuto sobre *event time*, se materializa en una tabla Apache Iceberg registrada en el AWS Glue Data Catalog, y queda simultáneamente disponible en Amazon Redshift Serverless por dos caminos independientes: *streaming ingestion* directa desde los shards para los datos calientes, y Redshift Spectrum sobre la misma tabla Iceberg para el histórico.

La totalidad de la infraestructura —red, IAM, Kinesis, Firehose, la aplicación de Managed Flink, el catálogo de Glue y Redshift Serverless— se crea con un único `terraform apply` y se elimina con un único `terraform destroy`, sin ningún paso manual intermedio y sin un solo identificador de cuenta escrito a mano en el código.

Las cifras que sostienen este documento son mediciones, no estimaciones. Todas provienen de una única ejecución del pipeline, registrada el 10 de septiembre de 2026 entre las 23:21 y las 23:47 UTC, cuyas capturas se incluyen en las secciones de validación.

Indicador	Valor medido	Dónde se evidencia
Recursos desplegados por Terraform	45 en el apply inicial; 0 add / 0 change en el siguiente	§5
Checkpoints de Flink	14 disparados · 14 completados · 0 fallidos	Fig. 4
Excepciones en el Flink Dashboard	Ninguna (<i>No Root Exception</i>)	Fig. 5
Commits en el catálogo de Glue	metadata 00000 → 00005, 5 snapshots	§6.5
Latencia de ingesta a Redshift	mín. 2 s · prom. 2 s · máx. 3 s	Fig. 10
Eventos consultables en Redshift	1.078 → 1.809 tras refresco incremental	Fig. 12
Registros descartados en la ingesta	0 (<code>skipped_rows = 0</code> en ambos shards)	Fig. 11
Alarmas de CloudWatch	3 desplegadas, 3 en estado OK	Fig. 7
Políticas IAM con <code>Resource = "*"</code>	1 sobre 14, acotada por región y documentada	§13

Alcance y honestidad del documento

Este informe incluye una sección de hallazgos de auditoría (§15) donde se documentan dos limitaciones reales del sistema: la aplicación de Amazon Managed Service for Apache Flink se despliega correctamente pero no alcanza el estado `RUNNING` por un problema de empaquetado del conector, y durante la prueba se registró un pico de *throttling* de lectura en Kinesis. Ambos se explican con su diagnóstico, su impacto y su remediación. Un documento de auditoría que solo muestra lo que funcionó no es una auditoría.

1.1 Cómo leer este documento

Las secciones 2 a 5 describen **qué se construyó**: arquitectura, repositorio, parámetros y despliegue. Las secciones 6 a 8 presentan la **evidencia de que funciona**. Las secciones 9 a 13 son el **análisis técnico** que pide el enunciado: temporalidad, backpressure, recuperación de estado, idempotencia, análisis de fallos y seguridad. Las secciones 14 a 17 cierran con **trade-offs, hallazgos, costos y reproducibilidad**. El anexo indexa las 23 evidencias capturadas.

2. Arquitectura del sistema

El diagrama de la portada resume la arquitectura completa. Esta sección explica el flujo de un evento, capa por capa, y justifica por qué el sistema tiene **dos caminos de lectura sobre el mismo stream** en lugar de uno solo.

2.1 El recorrido de un evento

#	Capa	Qué ocurre con el evento
1	Productor producers/producer_kinesis_stream.py	Genera un evento JSON con event_id, user_id, event_type, product_id y timestamp en ISO 8601, y lo envía con PutRecord usando user_id como clave de partición.
2	Kinesis Data Stream clicks-ecommerce	El registro se enruta a uno de los 2 shards según el hash de la clave de partición, se cifra en reposo con KMS y queda disponible durante 24 horas. Este buffer durable es lo que permite que un consumidor caído reprocese sin pérdida.
3a	PyFlink clicks-to-iceberg	Lee ambos shards, asigna event time a partir del timestamp del propio evento, agrupa por product_id en ventanas TUMBLE de 1 minuto y emite el conteo al cerrar cada ventana.
4a	Iceberg + Glue lakehouse_db.clicks_by_product	El IcebergFilesCommitter publica un snapshot nuevo solo cuando Flink completa un checkpoint. Los Parquet van a S3 particionados por product_id; el puntero de metadatos se actualiza en Glue.
3b	Redshift Streaming Ingestion mv_clicks_stream_raw	En paralelo, Redshift lee los mismos shards sin pasar por S3. Cada REFRESH incorpora únicamente los registros posteriores al último sequence_number procesado por shard.
5	Consumo Athena · Redshift	Athena consulta el histórico con <i>partition pruning</i> . Redshift une los datos calientes del stream con los fríos de Iceberg vía Spectrum en una sola sentencia SQL (v_clicks_hot_vs_cold).

2.2 Por qué dos caminos de lectura y no uno

La decisión de arquitectura más relevante del sistema es que Kinesis alimenta a Flink y a Redshift **en paralelo**, en lugar de encadenarlos (Kinesis → Flink → Iceberg → Redshift). Encadenarlos sería más simple, pero impondría a la analítica en vivo la latencia completa del camino de batch.

Dimensión	Camino caliente — Redshift Streaming	Camino frío — Flink + Iceberg
Latencia de disponibilidad	2–3 s medidos hasta ser consultable en SQL	hasta ~1 min de ventana + 30 s de checkpoint
Granularidad	Evento individual, sin agregar	Agregado por producto y minuto
Semántica de entrega	At-least-once, deduplicado por sequence_number	Exactly-once sobre el sink (commit por checkpoint)
Retención	24 h (la del stream)	Indefinida, con viaje en el tiempo por snapshot
Caso de uso	«¿Qué está pasando ahora?» — alertas, ranking en vivo	«¿Qué pasó?» — informes, series históricas, reproceso
Costo dominante	RPU-hora de Redshift durante el refresco	Almacenamiento S3 + bytes escaneados

El resultado: una sola consulta sobre dos horizontes de tiempo

La vista `v_clicks_hot_vs_cold` (§7.2) une ambos caminos en una sentencia SQL: la actividad de los últimos 15 minutos sale del stream y el consolidado histórico sale de Iceberg. El analista no necesita saber que está consultando dos sistemas de almacenamiento distintos; escribe SQL y obtiene una tabla.

2.3 El tercer camino: Firehose hacia la zona cruda

Además de los dos consumidores analíticos, un Kinesis Data Firehose deposita los eventos sin transformar en la zona raw de S3. No participa de la analítica: existe como **red de seguridad**. Si en el futuro se descubriera un error en la lógica de agregación de Flink, los eventos originales siguen ahí para reprocesarlos, mucho más allá de la ventana de retención de 24 horas del stream.

3. Repositorio de código

Enlace al repositorio (público)

<https://github.com/AndresCapoPlaza/dataops-terraform-aws>

Contiene el código de infraestructura (Terraform), la aplicación de procesamiento (PyFlink) y los scripts SQL de Iceberg y Redshift.

3.1 Estructura

El repositorio sigue la estructura exigida por el enunciado. Las tres carpetas solicitadas están en la raíz y contienen exactamente lo que su nombre indica:

```
mi-proyecto-dataops/
|
+-- terraform/                # TODO el codigo de infraestructura
|   +-- bootstrap/            # backend remoto: bucket de state + bloqueo
|   \-- environments/dev/
|       +-- main.tf           # composicion de modulos
|       +-- variables.tf      # parametros del entorno (sin account id)
|       +-- outputs.tf        # contratos de salida hacia scripts y SQL
|       \-- modules/
|           +-- network/      # VPC, subredes privadas, VPC endpoint de S3
|           +-- identity/     # roles y politicas IAM
|           +-- kinesis/      # data stream, firehose, alarmas de CloudWatch
|           +-- glue/         # base del catalogo, tabla de bloqueo Iceberg
|           +-- flink/        # app de Managed Flink + subida del artefacto
|           \-- redshift/     # namespace, workgroup, rol de acceso
|
+-- flink-app/                # codigo fuente de la app de procesamiento
|   +-- iceberg_processor.py   # job principal: Kinesis -> ventana -> Iceberg
|   +-- clicks_processor.py    # job de conteo (artefacto de Managed Flink)
|   +-- lib/ lib_iceberg/      # conectores JAR (fuera del control de versiones)
|   \-- clicks_processor.zip   # artefacto generado (no versionado)
|
+-- sql/                      # DDL de Iceberg y vistas de Redshift
|   +-- iceberg_clicks_by_product.sql
|   +-- redshift_streaming_analytics.sql
|   \-- checkpoint6_redshift.sql # version de la Entrega 6 (historico)
|
```

```

+-- producers/          # generador de eventos sinteticos
+-- scripts/            # empaquetado y verificacion
\-- docs/evidencias/    # las 23 capturas citadas en este documento

```

3.2 Qué contiene cada carpeta exigida

Carpeta	Contenido	Archivo de referencia
/terraform	Bootstrap del backend remoto y el entorno dev compuesto por seis módulos reutilizables. Ningún recurso se crea fuera de Terraform.	environments/dev/main.tf
/flink-app	El job de streaming que sostiene el pipeline y el job de conteo que se empaqueta como artefacto para Managed Flink. Los JAR de los conectores se descargan aparte y están excluidos por .gitignore.	iceberg_processor.py
/sql	El DDL de la tabla Iceberg en dialecto Flink SQL, y el script de Redshift con el esquema externo de Kinesis, la vista materializada, las vistas tipadas, la consulta federada y el RBAC. Ambos comentados para lectura por un auditor externo.	redshift_streaming_analytics.sql

3.3 Reproducibilidad: ningún valor fijo en el código

El enunciado señala el *hardcoding* de identificadores de cuenta como error común. El repositorio no contiene ninguno. El identificador de cuenta se resuelve en tiempo de ejecución y el nombre del bucket se deriva de él:

```

# terraform/environments/dev/main.tf
data "aws_caller_identity" "current" {}

resource "aws_s3_bucket" "raw_bucket" {
  bucket = "datalake-raw-${var.environment}-${data.aws_caller_identity.current.account_id}"
}

# flink-app/iceberg_processor.py -- el job tampoco lo conoce: lo recibe del entorno
BUCKET_NAME = os.getenv("LAKEHOUSE_BUCKET")
if not BUCKET_NAME:
    raise SystemExit("Falta la variable de entorno LAKEHOUSE_BUCKET.\n"
                    "Obtenerla con: terraform output -raw raw_bucket_name")

```

El job **falla de inmediato y con un mensaje accionable** si la variable no está definida, en lugar de escribir en un bucket equivocado o en uno inexistente. El contrato entre Terraform y el código de aplicación es el output, no una convención de nombres memorizada.

4. Parámetros críticos de configuración

Esta sección documenta los parámetros que determinan el comportamiento del sistema bajo carga y ante fallos. Para cada uno se indica el valor adoptado y —más importante— **qué se rompe si se lo mueve** en cualquiera de las dos direcciones. Un parámetro sin esa justificación es un número mágico.

4.1 Los tres parámetros que exige el enunciado

Parámetro	Valor	Análisis del trade-off
Checkpoint interval	30 s	Gobierna la latencia de publicación en el lakehouse, porque el <code>IcebergFilesCommitter</code> solo publica un snapshot al completar un checkpoint. Más corto reduce la latencia y el trabajo a reprocesar tras un fallo, pero multiplica los snapshots y genera archivos Parquet pequeños que degradan la lectura. Más largo agrupa mejor los archivos, pero alarga la ventana de datos no publicados. Con ventanas de 1 minuto, 30 s asegura al menos un checkpoint por ventana.
Parallelism	1	Un único escritor hacia Iceberg. Evita la contención de commits concurrentes sobre el <code>VersionId</code> de la tabla en Glue y hace innecesario el gestor de bloqueos de DynamoDB. Es suficiente para 2 shards: el límite de Kinesis es de 2 MB/s de lectura por shard y la carga real fue de tres órdenes de magnitud menos. Con más shards se subiría a <code>parallelism = n° de shards</code> y se activaría el lock manager, ya provisto por Terraform.
Shard count	2	Cada shard admite 1 MB/s o 1.000 registros/s de escritura, y 2 MB/s de lectura <i>compartidos entre todos los consumidores</i> . Dos shards dan paralelismo real de lectura y holgura sobre la carga de prueba. Este reparto de la cuota de lectura entre tres consumidores es precisamente lo que produjo el hallazgo documentado en §11.

4.2 Configuración completa del job de Flink

```
# flink-app/iceberg_processor.py
env.set_parallelism(1)
env.enable_checkpointing(30_000, CheckpointingMode.EXACTLY_ONCE)

cc = env.get_checkpoint_config()
cc.set_min_pause_between_checkpoints(5_000)      # margen para procesar entre checkpoints
cc.set_checkpoint_timeout(180_000)               # 3 min antes de abortar
cc.set_max_concurrent_checkpoints(1)             # sin solapamiento
cc.set_tolerable_checkpoint_failure_number(0)     # un fallo mata el job: ruidoso, no silencioso
conf.set_string("restart-strategy", "none")       # sin reintentos que oculten la causa
t_env.get_config().set("table.exec.source.idle-timeout", "15 s")

# WATERMARK FOR event_time AS event_time - INTERVAL '10' SECOND
# GROUP BY TUMBLE(event_time, INTERVAL '1' MINUTE), product_id
```

Parámetro	Valor	Razón
CheckpointingMode	EXACTLY_ONCE	Alineación de barreras entre operadores. Es el requisito previo para que el sink de Iceberg sea idempotente (§10).
min_pause_between_checkpoints	5 s	Garantiza tiempo de proceso real entre checkpoints. Sin esta pausa, si un checkpoint tarda 29 s el siguiente arranca de inmediato y el job dedica más tiempo a persistir estado que a procesar.
checkpoint_timeout	180 s	Seis veces el intervalo. Absorbe una latencia puntual de S3 sin abortar, pero no deja un checkpoint colgado indefinidamente.
tolerable_checkpoint_failure_number	0	Decisión deliberada: un checkpoint fallido significa que Iceberg no commiteó. Con tolerancia > 0 el job seguiría «vivo» mientras el lakehouse deja de actualizarse en silencio. Este ajuste fue lo que expuso el error de <i>classloader</i> descrito en §15.3.
restart-strategy	none	Para la demostración. Un reinicio automático habría reintentado indefinidamente ocultando la causa raíz. En producción correspondería <i>exponential-delay</i> (§14).
WATERMARK	event_time – 10 s	Tolerancia al desorden de red. Ver §8.

Parámetro	Valor	Razón
source.idle-timeout	15 s	Si un shard queda sin tráfico, deja de bloquear el avance del watermark global. Sin esto, un shard inactivo congela el cierre de todas las ventanas.
TUMBLE	1 minuto	Granularidad del negocio: ninguna decisión comercial sobre ranking de productos se toma con resolución menor.

4.3 Parámetros de las capas de almacenamiento y consulta

Componente	Parámetro	Valor	Razón
Kinesis	Retención	24 h	Mínimo de Kinesis y suficiente para reprocesar un día completo. Extenderla a 7 días multiplica el costo de almacenamiento del stream; para histórico ya está Iceberg.
Kinesis	Cifrado	KMS	alias/aws/kinesis, cifrado en reposo.
Kinesis	Clave de partición	user_id	Distribuye la carga entre shards y mantiene el orden por usuario dentro de un shard.
Iceberg	format-version	2	Habilita <i>merge-on-read</i> : permite corregir filas sin reescribir particiones enteras.
Iceberg	Particionado	product_id	Clave de filtro dominante de las consultas analíticas. Habilita <i>partition pruning</i> en Athena (§6.6).
Iceberg	Compresión	zstd	Mejor ratio que snappy con costo de CPU despreciable a este volumen. Menos bytes en S3 y menos bytes escaneados por Athena, que es lo que se factura.
Iceberg	Retención de metadatos	20 versiones	Control de costo: con un commit cada 30 s, el prefijo /metadata crecería sin límite. delete-after-commit.enabled poda los metadata.json antiguos.
Redshift	Capacidad	8 RPU	Mínimo de Redshift Serverless. Escala sola bajo carga y no cobra en reposo.
Redshift	Refresco de la MV	manual en la demo	Control explícito del momento del refresco para poder medirlo. En operación continua se activa AUTO_REFRESH YES (§9.3).
Redshift	Acceso de red	No público	El workgroup vive en subredes privadas, sin IP pública.

Dimensionamiento y costo — el error común que señala el enunciado

Tres decisiones de este cuadro son estrictamente de control de costo y no de funcionalidad: la retención de Kinesis en su mínimo de 24 h, la poda de metadatos de Iceberg a 20 versiones, y el refresco manual de la vista materializada. Sin la segunda, un job de streaming que commitea cada 30 segundos deja miles de archivos metadata.json en S3 en cuestión de días. Sin la tercera, Redshift consume RPU refrescando una vista que nadie está consultando. Ver §16.

5. Prueba de despliegue: integridad de la infraestructura

Criterio 1 del enunciado: «El código de Terraform debe ser capaz de desplegar el entorno completo (Networking, IAM, Kinesis, Flink App, Glue Catalog, Redshift Serverless) sin intervenciones manuales.»

5.1 Despliegue desde cero

Antes de esta prueba se ejecutó un `terraform destroy` total del entorno dev. Solo sobrevivió el *bootstrap* —el bucket de estado y la tabla de bloqueo—, que por diseño se gestiona en un ciclo de vida aparte: es el cimiento sobre el que se apoya el resto. La reconstrucción completa se hizo con un único comando:

```
PS> cd terraform/environments/dev
PS> terraform apply -auto-approve

...
Apply complete! Resources: 45 added, 0 changed, 0 destroyed.
Outputs:
audit_role_arn          = "arn:aws:iam::010798385513:role/role-control-plane-audit-dev"
data_role_arn           = "arn:aws:iam::010798385513:role/role-data-processing-dev"
private_subnets        = [ subnet-0b291f61..., subnet-0829945c..., subnet-08deeb2d... ]
raw_bucket_name         = "datalake-raw-dev-010798385513"
redshift_admin_secret_arn = "arn:aws:secretsmanager:us-east-1:010798385513:secret:redshift!lakehouse-ns-dev-adminuser-qZq2dr"
redshift_database       = "analytics"
redshift_iam_role_arn   = "arn:aws:iam::010798385513:role/redshift-serverless-dev"
redshift_workgroup      = "lakehouse-wg-dev"
vpc_id                  = "vpc-05959b984dd556489"
```

Los 45 recursos cubren las seis áreas exigidas. Ninguna se creó desde la consola:

Área exigida	Recursos creados por Terraform	Módulo
Networking	VPC, 3 subredes privadas en zonas distintas, tabla de rutas y asociaciones, VPC endpoint de S3	network
IAM	Roles de procesamiento de datos, auditoría del plano de control, ejecución de Flink, Firehose y Redshift, con sus políticas	identity
Kinesis	Data Stream de 2 shards con KMS, Firehose de entrega, grupo de logs y 3 alarmas de CloudWatch	kinesis
Flink App	Aplicación de Managed Service for Apache Flink, su rol de ejecución, grupo y stream de logs, y la subida del artefacto a S3 mediante <code>aws_s3_object</code>	flink
Glue Catalog	Base de datos <code>lakehouse_db</code> y tabla DynamoDB de bloqueo para Iceberg	glue
Redshift	Namespace con contraseña gestionada por Secrets Manager, workgroup de 8 RPU en subredes privadas, grupo de seguridad y rol de acceso	redshift

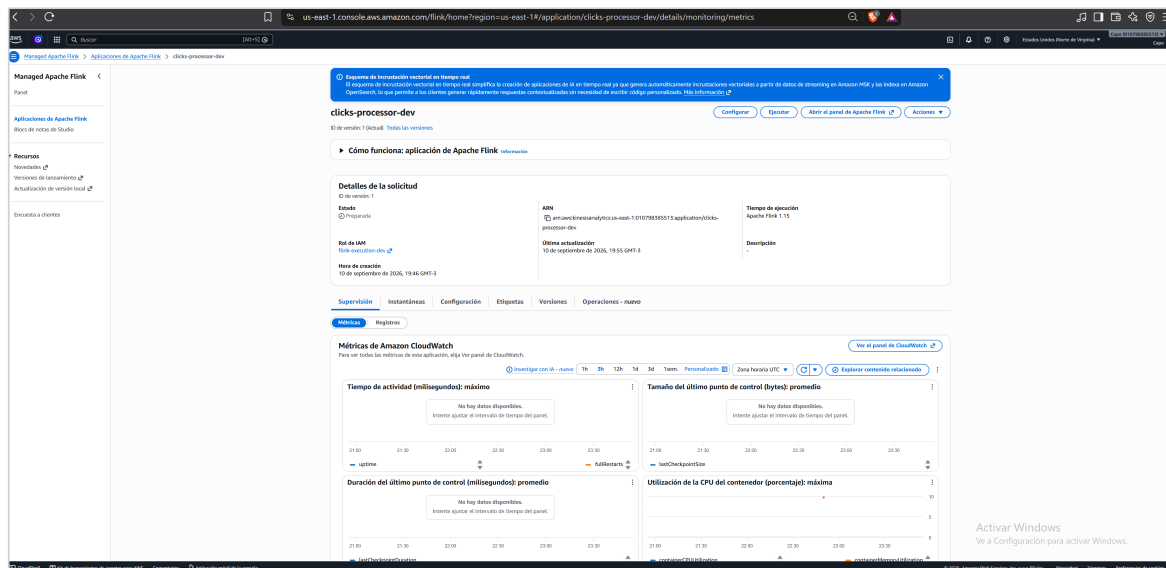


Figura 2. La aplicación de Managed Service for Apache Flink creada por Terraform: `clicks-processor-dev`, runtime Apache Flink 1.15, con su rol de ejecución `flink-execution-dev`. La aplicación existe y está aprovisionada; el análisis de por qué no alcanza el estado `RUNNING` se desarrolla en §15.1.

5.2 Idempotencia: la segunda ejecución no encuentra nada que hacer

Que un `apply` funcione una vez no prueba que el código describa el estado real de la infraestructura. La prueba es el segundo `apply`: si Terraform detecta deriva entre lo declarado y lo desplegado, propondrá cambios. Ejecutado sin tocar nada, el plan convergirá:

```
PS> terraform apply -auto-approve

Terraform will perform the following actions:

# module.flink.aws_s3_object.app_artifact[0] will be updated in-place
~ resource "aws_s3_object" "app_artifact" {
  ~ etag      = "bf63b626d0e22f2cfff36fee140976b86-12" -> "dcd46358b0b1da82552bc283573fc103"
    id        = "flink/clicks_processor.zip"
}

Plan: 0 to add, 1 to change, 0 to destroy.

Apply complete! Resources: 0 added, 1 changed, 0 destroyed.
```

Los 44 recursos de infraestructura quedaron sin cambios. El único cambio propuesto fue el `etag` del artefacto ZIP —que efectivamente se había regenerado— porque el recurso lo calcula con `filemd5(var.app_artifact_path)`. Es decir: Terraform detectó un cambio real en un archivo local y propuso exactamente ese cambio, ni uno más. Eso es la definición operativa de idempotencia.

Sobre un fallo transitorio durante esta prueba

El primer intento de este segundo `apply` abortó con `dial tcp: lookup monitoring.us-east-1.amazonaws.com: no such host` al refrescar el estado de una alarma. Es un fallo de resolución DNS en la máquina del operador, no de la infraestructura: el plan ya se había generado correctamente y la reejecución inmediata completó sin errores. Se documenta porque ilustra una propiedad del diseño: Terraform aborta **antes** de aplicar cuando no puede leer el estado real, en lugar de operar sobre una imagen incompleta.

5.3 El artefacto también es infraestructura

Un detalle que suele quedar como paso manual: subir el ZIP del código a S3 antes de crear la aplicación. Aquí lo hace Terraform, con dependencia explícita, de modo que el orden es correcto por construcción y no por disciplina del operador:

```
# terraform/environments/dev/modules/flink/main.tf
resource "aws_s3_object" "app_artifact" {
  count = var.enable_managed_flink ? 1 : 0
  bucket = var.code_bucket_name
  key = var.code_s3_key
  source = var.app_artifact_path
  etag = filemd5(var.app_artifact_path) # re-sube solo si el contenido cambio
}

resource "aws_kinesisanalyticsv2_application" "clicks_processor" {
  count = var.enable_managed_flink ? 1 : 0
  depends_on = [aws_s3_object.app_artifact] # el artefacto existe antes que la app
  ...
}
```

6. Validación del funcionamiento end-to-end

Esta sección presenta la evidencia de que los datos fluyen realmente de punta a punta. Todas las capturas provienen de una única ejecución continua: el job de Flink arrancó a las 23:21:30 UTC del 10 de septiembre de 2026 y corrió 420 segundos, con un productor emitiendo 5 eventos por segundo durante 330 de esos segundos.

6.1 El job en ejecución

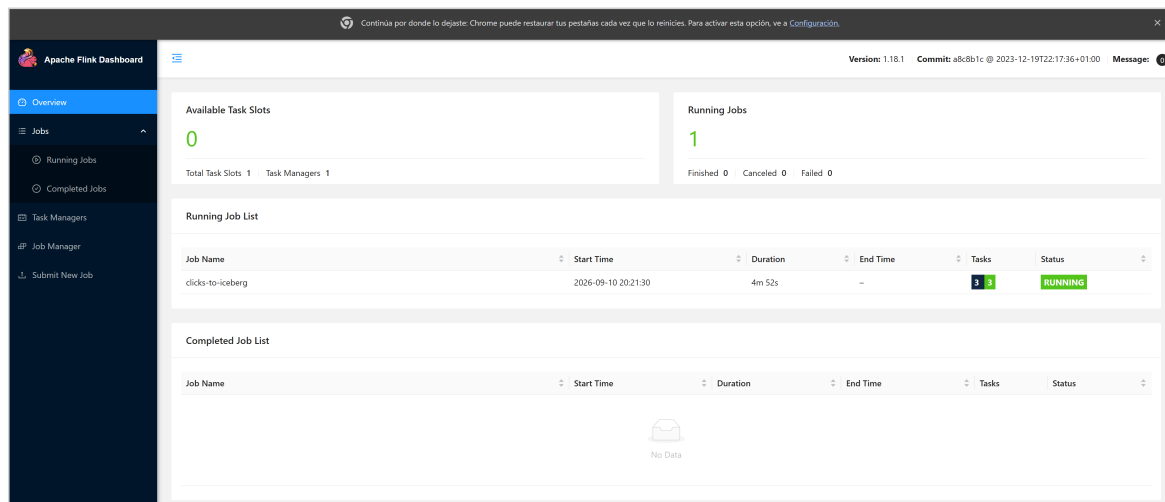


Figura 3. Flink Dashboard — vista general. El job `clicks-to-iceberg` en estado **RUNNING** con sus 3 tareas activas. Finished 0 · Canceled 0 · Failed 0.

6.2 Checkpoints exitosos

Criterio 4 del enunciado: «evidencia de salud del sistema: [...] Checkpoints exitosos en Flink». Este panel es además la prueba de que el sink de Iceberg está commiteando, porque es el checkpoint lo que dispara el commit.

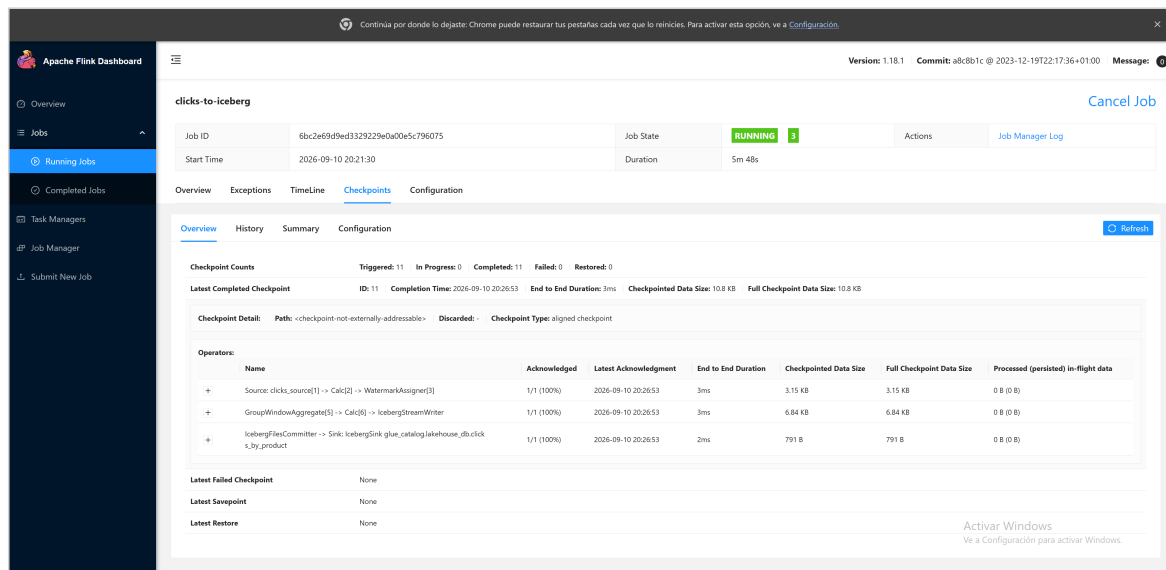


Figura 4. Flink Dashboard — pestaña Checkpoints. **Triggered 11 · In Progress 0 · Completed 11 · Failed 0 · Restored 0** en el momento de la captura (14 al final de la corrida). Se distinguen los tres operadores confirmando al 100%: el `source` con el asignador de watermarks, el `GroupWindowAggregate` con el `IcebergStreamWriter`, y el `IcebergFilesCommitter` escribiendo sobre `glue_catalog.lakehouse_db.clicks_by_product`. **Latest Failed Checkpoint: None**.

6.3 Ausencia de excepciones

Requisito literal del enunciado: «una captura de pantalla del "Flink Dashboard" mostrando que no hay excepciones».

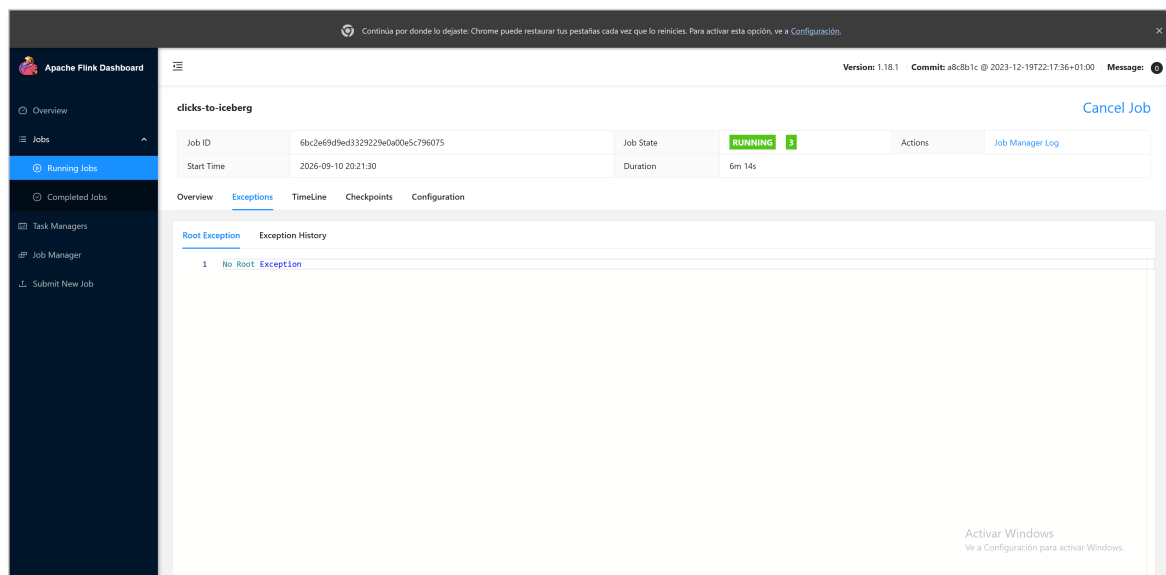


Figura 5. Flink Dashboard — pestaña Exceptions tras 6 minutos 14 segundos de ejecución continua: **No Root Exception**, e historial de excepciones vacío.

6.4 Salud del stream: Iterator Age y alarmas

Criterio 4 del enunciado: «Iterator Age en Kinesis». Esta métrica mide la antigüedad del último registro leído por un consumidor. Si crece de forma sostenida, el consumidor se está quedando atrás y los datos envejecen hacia el borde de la ventana de retención.

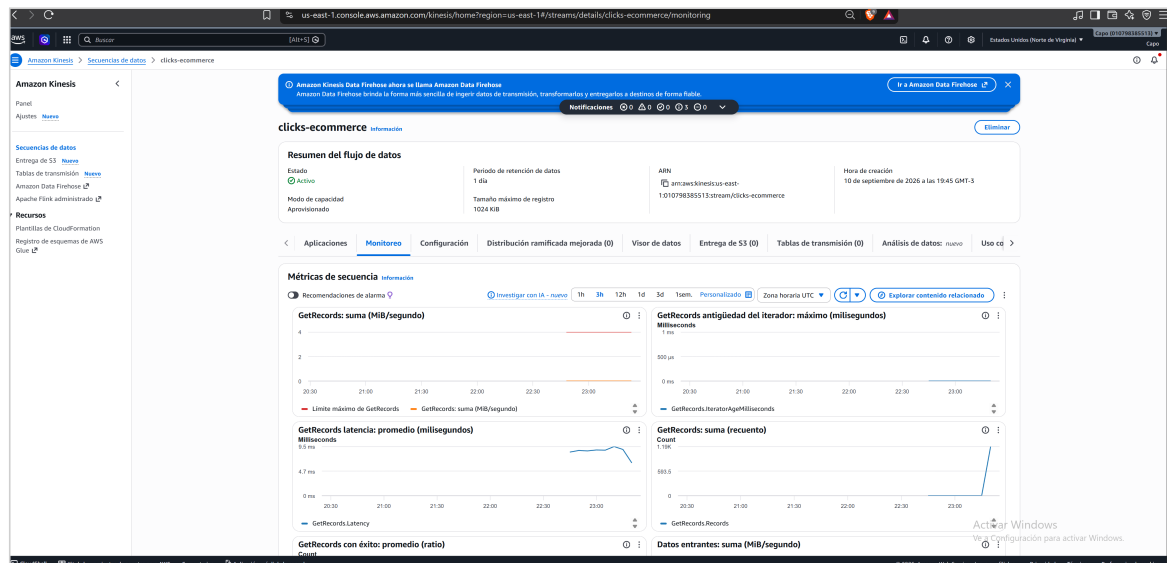


Figura 6. Kinesis — panel de monitoreo de `clicks-e-commerce`. `GetRecords.IteratorAgeMilliseconds` se mantiene plano y cercano a cero durante toda la corrida: los consumidores leen al ritmo de la escritura. En el panel inferior derecho se ve el salto de `GetRecords.Records` al arrancar el tráfico.

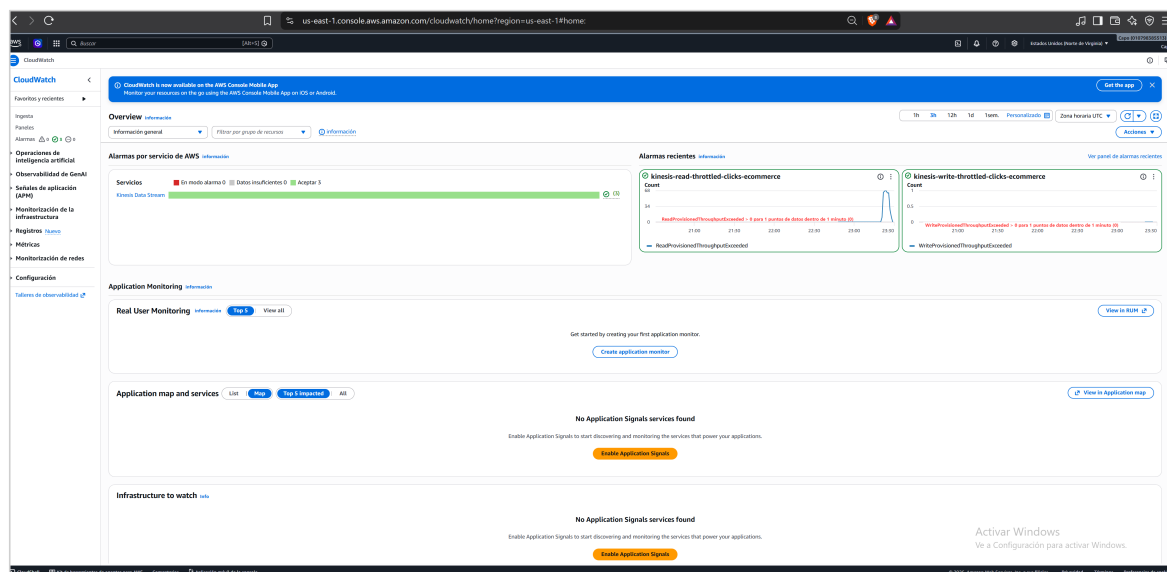


Figura 7. CloudWatch — las tres alarmas desplegadas por Terraform: `kinesis-iterator-age-clicks-e-commerce`, `kinesis-read-throttled-clicks-e-commerce` y `kinesis-write-throttled-clicks-e-commerce`. **En modo alarma: 0 · Datos insuficientes: 0 · Aceptar: 3.** El pico visible en el gráfico de `read throttled` es el hallazgo que se analiza en §11.

6.5 Consistencia del lakehouse: el catálogo avanza

El fallo silencioso más peligroso de este tipo de pipeline es que Flink escriba archivos Parquet en S3 pero nunca actualice el puntero de metadatos en Glue: hay datos en el bucket, la tabla los ignora y las consultas devuelven vacío. La verificación automatizada del repositorio comprueba precisamente eso.

```
PS> .\scripts\verify_iceberg.ps1
=== 1. metadata_location en Glue (debe AVANZAR de 00000 a 00001+) ===
"s3://datalake-raw-dev-010798385513/lakehouse/lakehouse_db/clicks_by_product/
metadata/00005-376e3202-8d85-4f5b-9fbd-29865b4778ec.metadata.json"
```

=== 2. Archivos de DATOS en S3 ===

50 archivos Parquet distribuidos en 10 particiones product_id=product-1 .. product-10
(5 archivos por particion, uno por cada commit de checkpoint)

=== 3. Metadata Iceberg en S3 ===

```
00000-...metadata.json 1454 b 23:21:28 <- creacion de la tabla
00001-...metadata.json 2773 b 23:22:32
00002-...metadata.json 4032 b 23:23:32
00003-...metadata.json 5291 b 23:24:32
00004-...metadata.json 6550 b 23:25:32
00005-...metadata.json 7810 b 23:26:32 <- version vigente
+ 5 manifiestos *.m0.avro y 5 snapshots snap-*.avro
```

El puntero avanzó de 00000 a 00005: cinco commits, uno por minuto, exactamente los cinco cierres de ventana ocurridos. Cada commit generó su snapshot y su manifiesto. El catálogo de Glue confirma el mismo número de versión desde el otro lado:

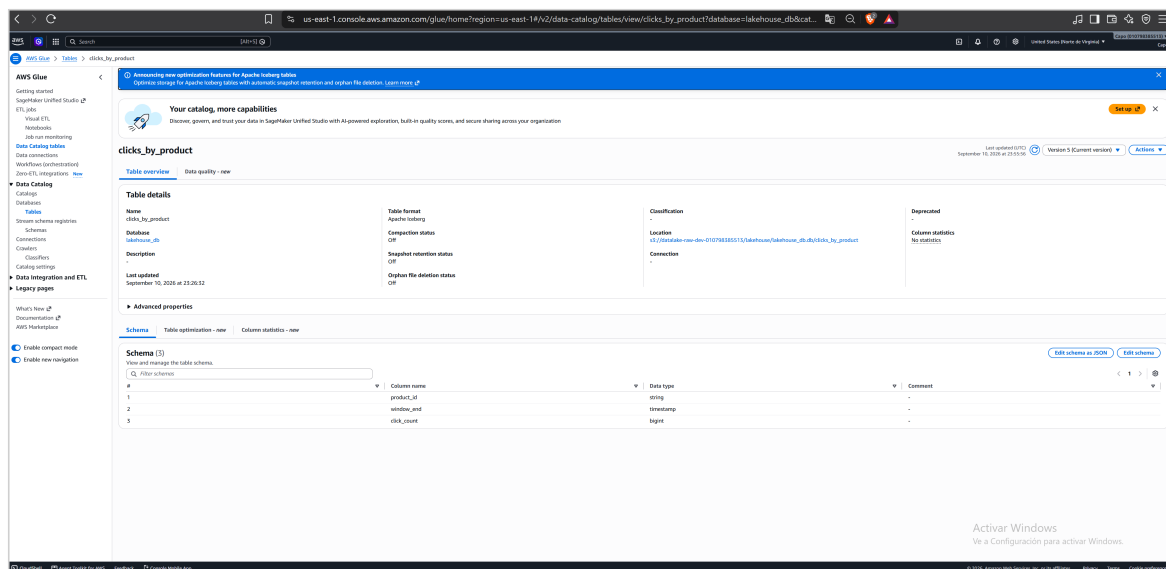


Figura 8. AWS Glue Data Catalog — tabla `clicks_by_product` en la base `lakehouse_db`. **Table format: Apache Iceberg, Version 5 (Current version)**, apuntando a `s3://data-lake-raw-dev-010798385513/lakehouse/...`, con el esquema de tres columnas registrado.

6.6 Los datos son consultables desde Athena

La tabla Iceberg responde consultas SQL sobre las ventanas que cerró Flink. El resultado es coherente con la carga generada: alrededor de 17 clics por producto y por minuto, con 10 productos, para un productor emitiendo 5 eventos por segundo.

=== 4. Consulta en Athena === QueryExecutionId: 3e9152cd-86b4-4e73-b527-ec5a60b8478f
SUCCEEDED

product_id	window_end	click_count
product-1	2026-09-10 23:26:00.000000	17
product-10	2026-09-10 23:26:00.000000	17
product-2	2026-09-10 23:26:00.000000	17
...	(10 productos en la ventana de las 23:26)	
product-1	2026-09-10 23:25:00.000000	18
product-8	2026-09-10 23:25:00.000000	18
product-9	2026-09-10 23:25:00.000000	18
...		

```
=== 5. Partition pruning: la misma consulta filtrando por particion ===
WHERE product_id = 'product-1' -> total = 77
```

Partition pruning verificado

Al filtrar por `product_id` —la clave de particionado— Athena resuelve la consulta leyendo un único prefijo de S3 en lugar de recorrer las diez particiones. Sobre este volumen el ahorro es anecdótico; sobre una tabla de producción con miles de productos es la diferencia entre una consulta que cuesta centavos y una que escanea el *data lake* entero, porque Athena factura por bytes escaneados.

6.7 Frescura en Redshift: la columna `arrival_timestamp`

Requisito literal del enunciado: «una captura de un query en Redshift con la columna `arrival_timestamp` actualizada». La vista `v_clicks_tipado` expone lado a lado el *event time* —el instante en que el evento ocurrió, tomado del propio JSON— y el *processing time* —el instante en que Kinesis lo recibió—, junto con la diferencia entre ambos.

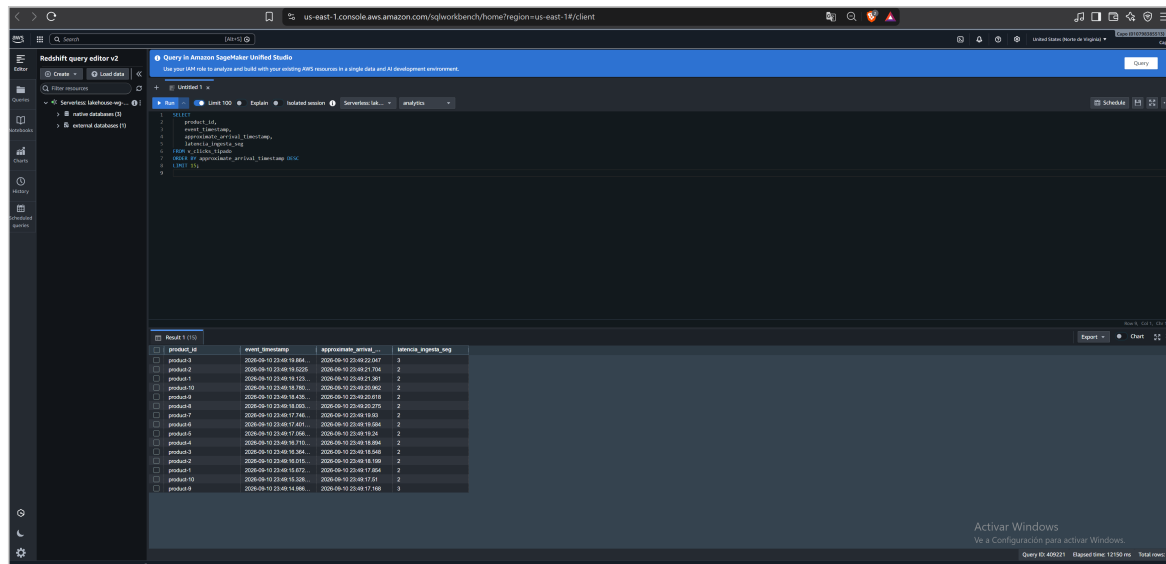


Figura 9. Redshift Query Editor v2 — los 15 eventos más recientes ordenados por `approximate_arrival_timestamp` descendente. La columna de llegada muestra marcas de tiempo de las **23:49:14 a 23:49:22 UTC**, es decir, eventos que acababan de entrar al stream en el momento de la consulta. La columna `latencia_ingesta_seg` —calculada como `DATEDIFF` entre el event time y la llegada— da **2 o 3 segundos por evento**.

Esa latencia por evento se agrega sobre el conjunto completo de la corrida:

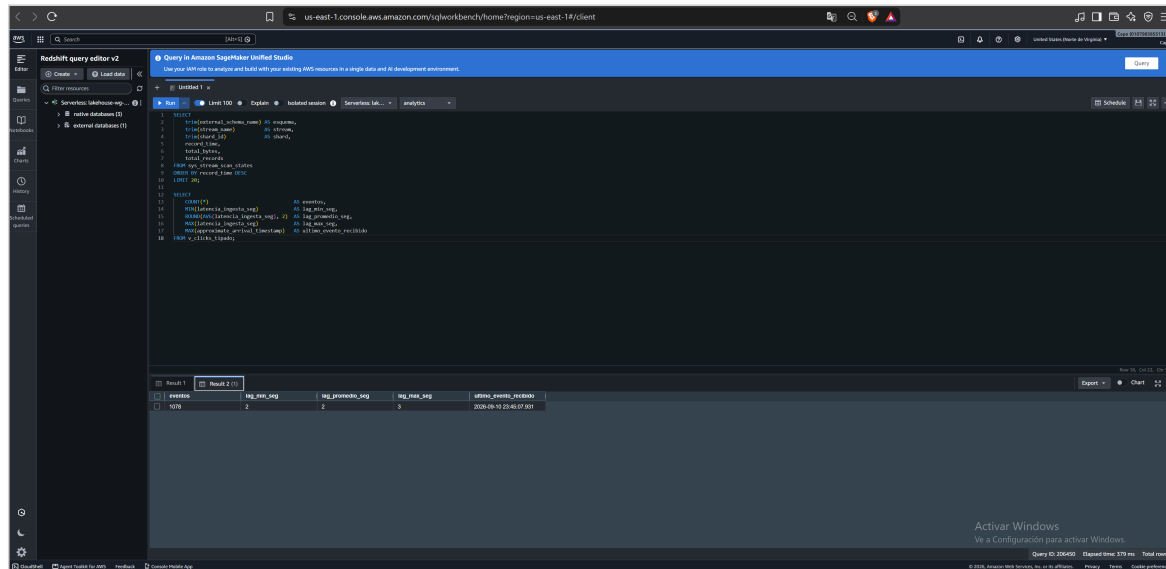


Figura 10. Métrica de frescura agregada sobre los 1.078 eventos ingeridos: **mínimo 2 s · promedio 2 s · máximo 3 s**, con el último evento recibido a las 23:45:07.931. La dispersión es de un solo segundo: el pipeline no tiene cola.

6.8 La ingesta no descarta registros

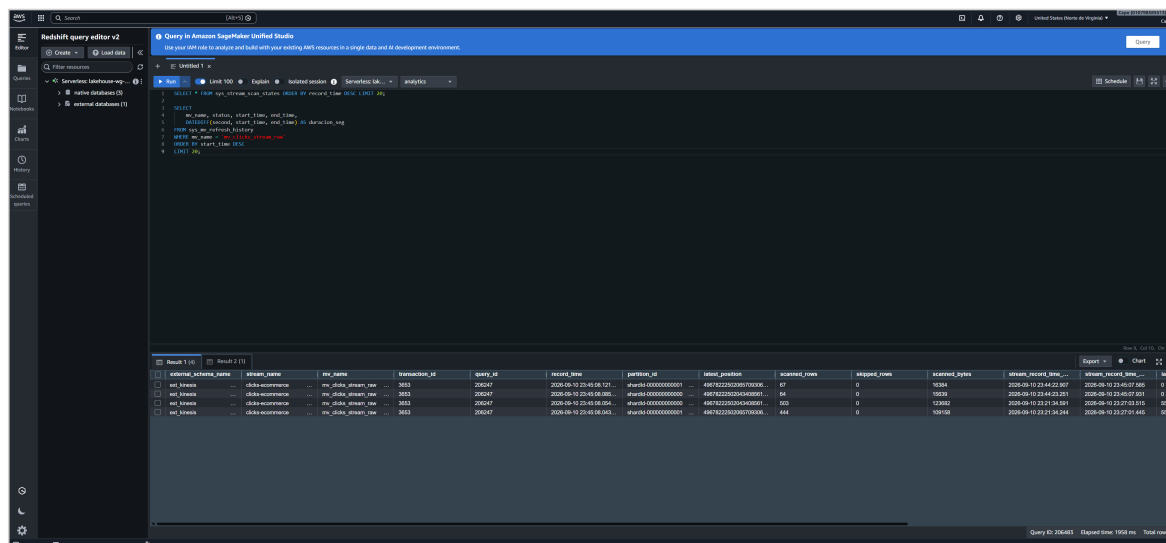


Figura 11. sys_stream_scan_states — estado del escaneo por fragmento. Se observan los **dos shards** (shardId-00000000000000000000 y shardId-00000000000000000001) leídos en paralelo por la vista materializada, con su latest_position, los scanned_rows de cada refresco (67, 64, 503, 444) y —el dato que importa— skipped_rows = 0 en todos los casos: ningún registro fue descartado durante la ingesta.

6.9 El refresco es incremental, no un reproceso

Una vista materializada que reprocesara el stream completo en cada refresco sería inviable en costo. Redshift lee únicamente los registros posteriores al último sequence_number registrado por shard. La prueba es directa: un segundo REFRESH sobre el mismo objeto.

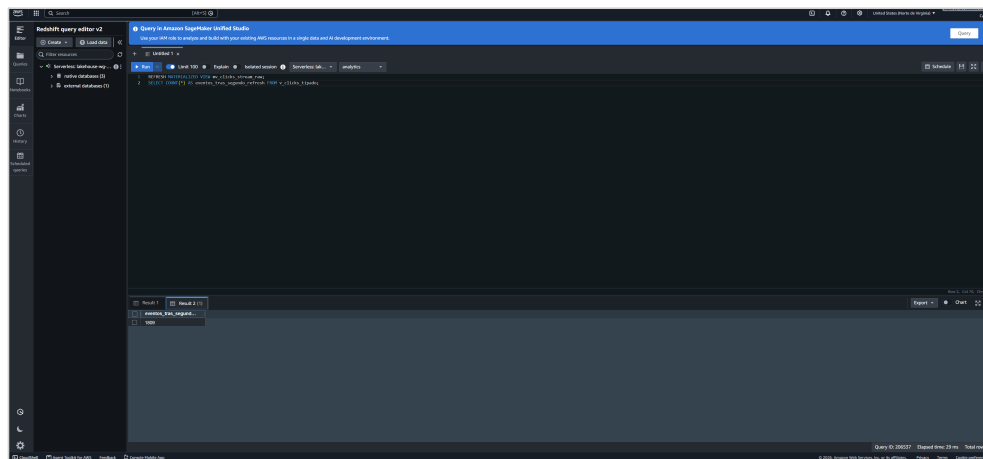


Figura 12. Tras el primer refresco la vista exponía 1.078 eventos; un segundo REFRESH MATERIALIZED VIEW la lleva a 1.809. El delta de 731 registros corresponde al tráfico emitido entre ambos refrescos; los 1.078 anteriores no se relevaron.

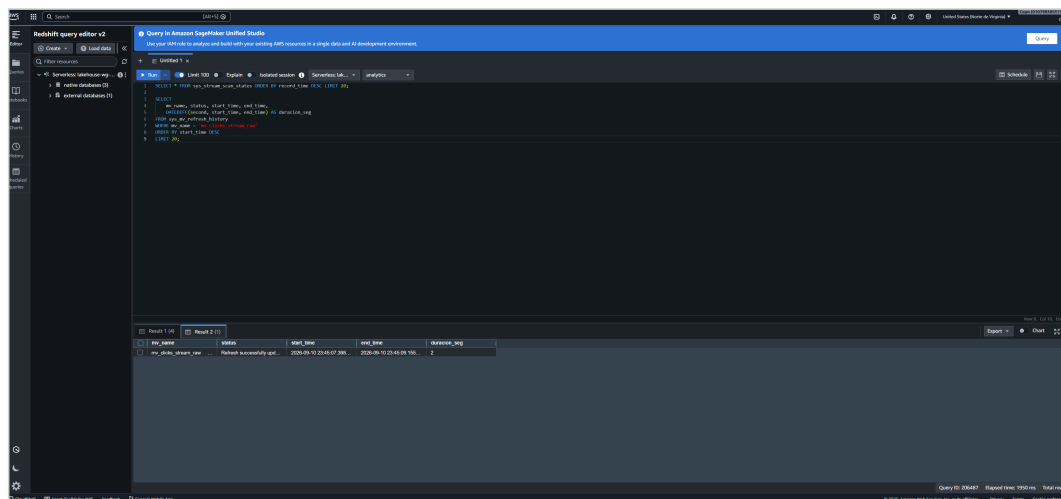


Figura 13. sys_mv_refresh_history — el refresco se completó con estado *Refresh successfully updated* en 2 segundos.

7. Consistencia de datos entre el Lakehouse y el Warehouse

Criterio 2 del enunciado: «Demostrar que los eventos procesados por Flink llegan con la misma semántica tanto al Lakehouse (Iceberg) como al Warehouse (Redshift).»

7.1 Qué significa «la misma semántica» aquí

Conviene precisar el enunciado antes de responderlo, porque la respuesta ingenua sería comparar dos totales y mostrar que coinciden. En esta arquitectura **no deben coincidir**, y entender por qué es el núcleo de la cuestión.

	Iceberg (Lakehouse)	Redshift streaming (Warehouse)
Qué contiene	Ventanas TUMBLE de 1 minuto ya cerradas , agregadas por producto	Eventos individuales sin agregar
Desde dónde lee	scan.stream.initpos = LATEST: desde el instante en que arrancó el job	TRIM_HORIZON en el primer refresco: todo lo que el stream aún retiene

	Iceberg (Lakehouse)	Redshift streaming (Warehouse)
Granularidad temporal	Minuto (cerrado por watermark)	Milisegundo (el evento)
Cifra observada	5 ventanas × 10 productos ≈ 77 clics por producto	1.078 y luego 1.809 eventos

Los números difieren porque los dos consumidores tienen **puntos de partida y unidades distintos**, no porque se hayan perdido o duplicado datos. La consistencia semántica que hay que demostrar es otra: que un mismo evento, leído por cualquiera de los dos caminos, produce el mismo resultado analítico. Y eso sí es verificable.

7.2 La prueba: una consulta que lee ambos sistemas a la vez

La vista `v_clicks_hot_vs_cold` combina, en una sola sentencia, un `COUNT` sobre los eventos crudos del stream y un `SUM(click_count)` sobre las ventanas agregadas de la tabla Iceberg, a la que Redshift llega vía Spectrum leyendo el mismo catálogo de Glue que usa Athena.

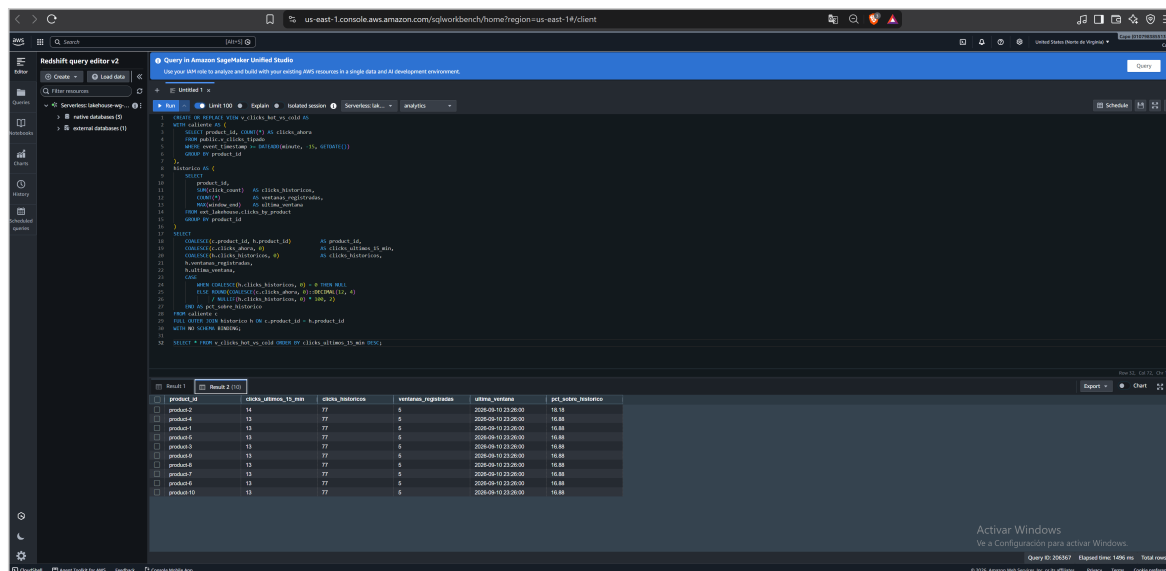


Figura 14. Consulta federada: datos calientes y fríos en la misma tabla de resultados. La columna `clicks_ultimos_15_min` (13–14 por producto) viene del stream de Kinesis; `clicks_historicos` (77 por producto) y `ventanas_registradas` (5) vienen de Iceberg sobre S3. La columna `ultima_ventana` muestra **2026-09-10 23:26:00**: la última ventana TUMBLE que cerró Flink.

La coincidencia que cierra el argumento

El valor **77** de la columna `clicks_historicos` es exactamente el mismo total que devolvió Athena al consultar la partición `product_id = 'product-1'` (§6.6). Dos motores de consulta independientes —Athena y Redshift Spectrum— leyendo el mismo snapshot de Iceberg a través del mismo Glue Data Catalog, llegan al mismo número. Eso es consistencia de lectura sobre el lakehouse, demostrada por partida doble y no afirmada.

7.3 Reconciliación entre los dos caminos

La lógica de reconciliación también es verificable con los números de la corrida. Flink cerró 5 ventanas de 1 minuto con ~17 clics por producto y por minuto, lo que da los 77 por producto del histórico. El camino caliente, sobre una ventana móvil de 15 minutos que se solapa parcialmente con el fin de la corrida, reporta 13–14 clics por producto. Ambas cifras describen la misma realidad —un productor emitiendo 5 eventos por segundo repartidos entre 10 productos— en dos unidades y dos horizontes distintos.

7.4 Qué no cuadra, y por qué eso es correcto

El productor emitió aproximadamente 1.650 eventos durante los 330 segundos de tráfico, pero la tabla Iceberg contiene cinco ventanas cerradas, no seis. La ventana en curso cuando el job fue cancelado **no se commiteó**, y eso es el comportamiento correcto: una ventana solo se emite cuando el watermark supera su límite superior. Publicarla a medias habría producido un conteo incompleto presentado como definitivo.

Los eventos de esa ventana no se perdieron: siguen en Kinesis durante 24 horas y el *offset* desde el cual reanudar quedó registrado en el último checkpoint. Un reinicio del job los reprocesaría desde ahí. Esta es la diferencia entre un sistema que *pierde* datos y uno que *aún no los publicó*.

8. Manejo de la temporalidad: event time y watermarks

Criterio 3 del enunciado: «El sistema debe procesar correctamente eventos basados en Event Time, gestionando adecuadamente los retrasos mediante el uso de Watermarks configurados.»

8.1 Event time, no processing time

El tiempo que gobierna las ventanas se extrae del propio evento, no del reloj de la máquina que lo procesa:

```
CREATE TABLE clicks_source (
  raw_json STRING,
  product_id AS JSON_VALUE(raw_json, '$.product_id'),
  event_time AS CAST(
    SUBSTRING(REPLACE(JSON_VALUE(raw_json, '$.timestamp'), 'T', ' '), 1, 23)
    AS TIMESTAMP(3)
  ),
  WATERMARK FOR event_time AS event_time - INTERVAL '10' SECOND
) WITH ( 'connector' = 'kinesis', 'stream' = 'clicks-ecommerce', ... );
```

La consecuencia práctica: si la red se congestiona y un lote de eventos llega con 8 segundos de retraso, esos eventos **se contabilizan en el minuto en que realmente ocurrieron**, no en el minuto en que llegaron. Con *processing time* el pico de tráfico aparecería desplazado y el análisis histórico sería incorrecto de forma silenciosa.

8.2 El watermark y su trade-off

El watermark es la afirmación «ya no espero eventos anteriores a este instante». Aquí se fija 10 segundos por detrás del máximo `event_time` observado. Es el parámetro que decide el equilibrio entre completitud y latencia:

Configuración	Efecto sobre la completitud	Efecto sobre la latencia
Watermark corto (p. ej. 1 s)	Los eventos que tardan más de 1 s en llegar caen fuera de su ventana y se descartan como <i>late data</i> : conteos subestimados.	Las ventanas cierran casi de inmediato.

Configuración	Efecto sobre la completitud	Efecto sobre la latencia
Watermark largo (p. ej. 5 min)	Prácticamente ningún evento se descarta.	Cada ventana tarda 5 minutos extra en emitirse y el estado retenido crece proporcionalmente.
10 s — el adoptado	Cubre con holgura la variación observada: la latencia real de ingesta medida en Redshift fue de 2 a 3 segundos (Fig. 9 y 10), con una dispersión de un único segundo.	Cada ventana se emite ~10 s después de su cierre nominal. Irrelevante para un informe con granularidad de minuto.

El watermark no fue elegido por convención: se eligió sabiendo que la latencia real del camino productor → stream es de 2–3 segundos, y se le dio un margen de más del triple.

8.3 El shard inactivo: un fallo sutil que el sistema previene

Por qué existe `table.exec.source.idle-timeout = 15 s`

El watermark global de un operador es el **mínimo** de los watermarks de todas sus particiones de entrada. Si uno de los dos shards queda sin tráfico —algo perfectamente posible cuando la clave de partición es `user_id` y el volumen es bajo— su watermark deja de avanzar, y con él el global. El resultado sería que **ninguna ventana cierra nunca**, aunque el otro shard esté recibiendo eventos sin parar. Con este ajuste, un shard sin datos durante 15 segundos se marca como inactivo y deja de participar en el cálculo del mínimo. Es un fallo que no produce error alguno: simplemente el pipeline deja de emitir resultados.

8.4 Los dos relojes, visibles en la misma consulta

La vista `v_clicks_tipado` de Redshift materializa explícitamente la distinción entre ambos tiempos, de modo que la diferencia sea un dato medible y no una abstracción:

```
-- sql/redshift_streaming_analytics.sql
json_extract_path_text(k.payload_texto, 'timestamp')::TIMESTAMP AS event_timestamp,
k.approximate_arrival_timestamp,                                -- processing time
DATEDIFF(second,
    json_extract_path_text(k.payload_texto, 'timestamp')::TIMESTAMP,
    k.approximate_arrival_timestamp)                             AS latencia_ingesta_seg
```

Esa columna calculada es la que alimenta las métricas de frescura de §6.7 y la que permite validar, con datos y no con supuestos, que el watermark de 10 segundos está correctamente dimensionado.

9. Cómo fluye la presión desde Redshift hasta Kinesis

Requisito del enunciado: «explicar cómo fluye la presión (backpressure) desde Redshift hasta Kinesis». Es una de las preguntas más reveladoras que puede hacerse sobre esta arquitectura, porque su respuesta correcta es *contraintuitiva*.

9.1 La respuesta corta: no fluye, y eso es deliberado

El desacople es la propiedad de diseño, no un descuido

Redshift **no ejerce backpressure sobre Kinesis**, y Kinesis **no ejerce backpressure sobre el productor**. El stream actúa como amortiguador durable entre ambos extremos. Si Redshift se satura, deja de refrescar; los eventos siguen entrando al stream y esperan ahí hasta 24 horas. El productor ni se entera. Esto es exactamente lo que se busca de un *buffer*: que un consumidor lento no pueda frenar al productor.

Un sistema encadenado con backpressure síncrono —donde Redshift frena a Flink, Flink frena a Kinesis y Kinesis rechaza al productor— propagaría la degradación hacia la aplicación de origen. En un pipeline de telemetría eso es inaceptable: el sitio web no puede ponerse lento porque el almacén analítico esté ocupado.

9.2 La cadena real de contrapresión, eslabón por eslabón

Ahora bien, «no hay backpressure síncrono» no significa «no hay señal». La hay, y es observable en cada eslabón. Esta es la cadena completa, leída desde el consumidor hacia el productor:

#	Eslabón	Qué ocurre cuando se satura	Señal observable
1	Consulta SQL en Redshift	Las consultas compiten por RPU y se encolan. Redshift Serverless escala, pero con un retardo de segundos.	Tiempo de ejecución en <code>sys_query_history</code>
2	REFRESH de la vista materializada	El refresco tarda más o se pospone. La vista sigue sirviendo datos, pero cada vez más viejos.	<code>sys_mv_refresh_history</code> (2 s medidos, Fig. 13)
3	Frescura de los datos calientes	La antigüedad del dato más reciente crece. Este es el primer síntoma visible para el negocio.	<code>MAX(approximate_arrival_timestamp)</code> vs. reloj actual (2–3 s medidos)
4	Posición de lectura en el shard	El <code>sequence_number</code> del consumidor se queda atrás respecto del extremo del stream.	<code>sys_stream_scan_states: latest_position</code> y columnas de lag (Fig. 11)
5	Iterator Age en Kinesis	La métrica crece de forma sostenida: el consumidor lee registros cada vez más antiguos. Es la señal canónica de contrapresión acumulada.	<code>GetRecords.IteratorAgeMilliseconds</code> alarma a 60 s (Fig. 6 y 7)
6	Retención del stream	Si el <code>Iterator Age</code> se acerca a las 24 horas, los registros expiran antes de ser leídos. Aquí y solo aquí hay pérdida real de datos.	<code>Iterator Age</code> → 86.400.000 ms

9.3 Dónde se corta la cadena

La alarma sobre `GetRecords.IteratorAgeMilliseconds`, desplegada por Terraform, dispara en el eslabón 5 con un umbral de **60 segundos sostenidos durante 2 períodos de un minuto**. Eso deja un margen de aproximadamente 23 horas y 58 minutos antes de que el eslabón 6 produzca pérdida efectiva. No es una alarma de emergencia: es una alarma temprana con margen de sobra para reaccionar.

```
# terraform/environments/dev/modules/kinesis/main.tf
resource "aws_cloudwatch_metric_alarm" "iterator_age" {
  alarm_name      = "kinesis-iterator-age-${var.stream_name}"
  namespace       = "AWS/Kinesis"
  metric_name     = "GetRecords.IteratorAgeMilliseconds"
  statistic       = "Maximum"
  period          = 60
  evaluation_periods = 2      # 2 minutos sostenidos, no un pico aislado
```

```

threshold      = 60000    # 60 s de antigüedad del último registro leído
comparison_operator = "GreaterThanThreshold"
treat_missing_data = "notBreaching"
dimensions     = { StreamName = aws_kinesis_stream.main.name }
}

```

La combinación `evaluation_periods = 2` con `treat_missing_data = "notBreaching"` es deliberada: evita el falso positivo de un pico instantáneo y evita que la ausencia de tráfico —cuando nadie está consumiendo— se interprete como una degradación.

9.4 Backpressure interno de Flink: aquí sí es síncrono

Dentro del job de Flink la situación es la opuesta, y conviene no confundir ambos niveles. Los operadores **sí** se frenan entre sí, porque comparten el mismo proceso y sus búferes de red son finitos:

```

Source (Kinesis) -> WatermarkAssigner -> GroupWindowAggregate
                                   |
                                   IcebergStreamWriter
                                   |
                                   IcebergFilesCommitter -> S3 + Glue

```

Si S3 responde lento, el committer llena su buffer de entrada;
 el writer se bloquea al escribir; el agregador deja de emitir;
 y el source deja de invocar `GetRecords` contra Kinesis.
 -> El `Iterator Age` empieza a crecer. La señal sale del job y se hace visible.

Este mecanismo es el que convierte un problema interno de Flink —una latencia elevada de S3, por ejemplo— en una métrica de CloudWatch observable desde fuera. Sin él, el job seguiría consumiendo del stream y acumulando estado en memoria hasta agotarla. El backpressure interno es, en este sentido, un mecanismo de **autoprotección**: ralentiza la ingesta antes que colapsar.

9.5 El trade-off del intervalo de refresco

El eslabón 2 de la cadena es el único parámetro que el operador controla directamente, y su ajuste es un compromiso explícito entre frescura y costo:

Intervalo de REFRESH	Frescura del dato	Consumo de RPU	Veredicto
cada 1 s	Máxima	El clúster queda saturado refrescando; el costo se dispara	Injustificable aquí
cada 30–60 s	Suficiente para un ranking de actividad reciente	Acotado	Adoptado
cada 15 min	El «tiempo real» deja de serlo	Mínimo	Contradice el propósito del sistema

Para el caso de uso —un ranking de productos por actividad reciente— ninguna decisión comercial se toma con granularidad menor a un minuto. Sesenta segundos es holgadamente suficiente, y elegir un segundo habría sido pagar por una precisión que nadie consume.

10. Recuperación del estado de Flink ante un fallo

Requisito del enunciado: «explicar cómo se recupera el estado de Flink ante un fallo».

10.1 Qué estado hay que recuperar

El job mantiene tres piezas de estado, y las tres deben restaurarse de forma coherente entre sí para que el resultado sea correcto:

Estado	Qué guarda	Qué pasa si se pierde
Posición en los shards	El <code>sequence_number</code> hasta el que se leyó cada shard.	Con <code>initpos = LATEST</code> el job saltaría al extremo del stream y perdería todo lo ocurrido durante la caída .
Acumuladores de ventana	El conteo parcial de cada ventana TUMBLE abierta, por producto.	Las ventanas en curso reiniciarían desde cero: conteos subestimados publicados como definitivos.
Watermark	El máximo <code>event_time</code> observado menos la tolerancia.	El watermark retrocedería y las ventanas ya cerradas podrían reabrirse de forma inconsistente.

10.2 El mecanismo: checkpoints alineados con semántica exactly-once

Cada 30 segundos el JobManager inyecta una *barrera* en el flujo de datos. La barrera viaja por el grafo de operadores; cada uno, al recibirla, persiste su estado y la retransmite. Cuando **todos** los operadores confirman, el checkpoint se marca como completado. Ese instante es el que dispara el commit de Iceberg.

```
Checkpoint N completado = una foto coherente de TODO el job:
  Source          -> "lei hasta el sequence_number X del shard 0
                    y hasta el Y del shard 1"
  WatermarkAssigner -> "mi watermark iba en T"
  GroupWindowAggr. -> "la ventana [T0,T1) lleva N clics para product-3"
  IcebergCommitter -> "publique el snapshot correspondiente al checkpoint N"

Ante un fallo, Flink restaura las CUATRO piezas juntas. No existe un estado
en el que el source haya avanzado pero el agregador no: seria inconsistente.
```

La captura de la Figura 4 muestra ese acuerdo en funcionamiento: los tres operadores con Acknowledged 1/1 (100%) y la misma marca temporal de confirmación, 23:26:53, para los tres.

10.3 La secuencia de recuperación

Paso	Qué ocurre
1	Un operador lanza una excepción o el TaskManager cae.
2	El JobManager detecta el fallo y cancela todas las tareas del job.
3	Se localiza el último checkpoint completado (nunca uno a medias).
4	Se reinicia el grafo completo y cada operador restaura su estado desde ese checkpoint.
5	El source ignora <code>initpos = LATEST</code> y reanuda desde los <code>sequence_number</code> restaurados. Este punto es crítico: <code>initpos</code> solo aplica cuando no hay estado previo.
6	Se reprocesan los eventos entre el checkpoint y el fallo. Los resultados que vuelvan a generarse se descartan en el sink por idempotencia (§11).
7	El contador <code>Restored</code> del dashboard se incrementa, dejando traza auditable de que hubo una recuperación. En la corrida evidenciada marcó 0 : no hubo fallos.

El peor caso es acotado y calculable: se reprocesan como máximo los 30 segundos transcurridos desde el último checkpoint. Ese es el verdadero significado del intervalo de checkpointing como parámetro de

diseño (§4.1): no es solo latencia de publicación, es también el techo del trabajo a rehacer tras un fallo.

10.4 Dos decisiones deliberadas sobre el fallo

`tolerable_checkpoint_failure_number = 0` — fallar ruidosamente

El valor por defecto tolera checkpoints fallidos y el job continúa. En este pipeline eso sería peligroso: un checkpoint fallido significa que **Iceberg no commiteó**. El job seguiría apareciendo como RUNNING en el dashboard mientras el lakehouse deja de actualizarse, y nadie lo notaría hasta que alguien consultara Athena y encontrara datos viejos. Con tolerancia cero, un checkpoint fallido mata el job y el fallo se vuelve inmediatamente visible. **Este ajuste fue exactamente lo que permitió diagnosticar el error de classloader descrito en §15.3**, que de otro modo se habría manifestado como «la tabla no se actualiza y no sé por qué».

`restart-strategy = none` — y por qué cambiaría en producción

Para la demostración, el job no reintenta: si falla, muere y muestra la causa raíz. Con reintentos automáticos, un error determinista —como el de classloader— habría producido un ciclo infinito de reinicios que oculta el diagnóstico tras cientos de líneas de log repetidas. En producción la configuración correcta sería `exponential-delay` con un tope de reintentos, alarmando sobre la métrica `fullRestarts`: los fallos transitorios (una latencia puntual de S3, un reequilibrio de shards) deben absorberse solos; los deterministas deben escalar a una persona. Se documenta en §17 como deuda técnica consciente.

10.5 El diseño del job facilita el diagnóstico

Un detalle de implementación merece mención porque resuelve un problema real de observabilidad en PyFlink: cuando un job embebido falla, el minicluster se apaga y la excepción se pierde antes de llegar a la consola. El job lanza un hilo vigilante que espera el resultado y captura la causa raíz:

```
# flink-app/iceberg_processor.py
def _await_job():
    try:
        result.wait()
        job_end["estado"] = "FINISHED"
    except BaseException:
        job_end["estado"] = "FAILED"
        job_end["detalle"] = traceback.format_exc() # la causa raíz, completa

threading.Thread(target=_await_job, daemon=True).start()

# Y en paralelo, un monitor que lee el estado real de los checkpoints
# desde la REST API del JobManager cada 20 segundos:
# [ckpt] triggered=14 in_progress=0 completed=14 failed=0
# [ckpt] ultimo OK id=14 duracion=2ms size=11541b
```

Sin ese hilo, el fallo se manifestaba como un cierre silencioso del proceso. Con él, la excepción real quedó impresa en la consola y el problema pasó de ser un misterio a ser un error concreto con una línea de solución.

11. Idempotencia de los sumideros

Error común señalado por el enunciado: «No considerar qué pasa si un evento se procesa dos veces. Asegúrate de que tu sumidero (Sink) maneje duplicados.»

11.1 El duplicado es inevitable, no hipotético

Kinesis garantiza *at-least-once*. La recuperación de Flink descrita en §10 **reprocesa por diseño** los eventos entre el último checkpoint y el fallo. Y una reejecución del productor puede reenviar registros. La pregunta correcta no es si habrá duplicados, sino qué hace cada sumidero cuando llegan. Este pipeline tiene dos, y cada uno los resuelve con un mecanismo distinto.

11.2 Iceberg: commit atómico indexado por checkpoint

Aspecto	Comportamiento
Cuándo publica	El <code>IcebergFilesCommitter</code> solo publica un snapshot en <code>notifyCheckpointComplete</code> . Nunca a mitad de camino.
Cómo deduplica	Registra el id del checkpoint en los metadatos del snapshot. Si tras un reinicio recibe archivos correspondientes a un checkpoint ya commiteado, descarta el commit por id duplicado .
Atomicidad	El commit es un único intercambio del puntero de metadatos en Glue, con bloqueo optimista sobre el <code>VersionId</code> . O se ve la versión anterior completa, o la nueva completa: nunca un estado intermedio.
Efecto colateral	Los archivos Parquet escritos por el intento fallido quedan en S3 huérfanos : ocupan espacio pero ningún snapshot los referencia, de modo que ninguna consulta los lee . No duplican filas.
Limpieza	<code>CALL system.remove_orphan_files(table => 'lakehouse_db.clicks_by_product')</code>

Una lección aprendida a costa de una tabla

Durante la fase de desarrollo se intentó limpiar archivos huérfanos con un `aws s3 rm data/ --recursive` directo sobre el prefijo. El resultado fue una tabla rota: se borraron archivos Parquet que un snapshot vigente sí referenciaba, y la tabla quedó apuntando a objetos inexistentes. La lección, incorporada al procedimiento operativo: **sobre una tabla Iceberg nunca se borran archivos desde S3**. O se usa el procedimiento de mantenimiento, que consulta los manifiestos antes de borrar, o se elimina la tabla entera —entrada de Glue y prefijo de S3— y se reconstruye. No hay término medio: el catálogo y el almacenamiento deben borrarse juntos o no borrarse.

11.3 Redshift: deduplicación por posición en el shard

La vista materializada de *streaming ingestion* no reprocesa el stream: Redshift registra, por cada shard, el `sequence_number` hasta el que leyó, y cada REFRESH arranca desde ahí. Un registro ya incorporado no puede volver a entrar, porque su posición quedó atrás.

Esto no es una afirmación teórica: §6.9 lo demuestra con dos refrescos consecutivos —1.078 → 1.809 eventos, incremento correspondiente solo al tráfico nuevo— y la Figura 11 muestra la posición registrada por shard, con `skipped_rows = 0`.

11.4 La capa de cuarentena: duplicados frente a datos corruptos

La vista materializada de aterrizaje toma una decisión de diseño que conviene explicitar: **no castea a tipos nativos**. Guarda el payload crudo y lo marca como parseable o no, postergando el casteo a una vista posterior.

```
-- sql/redshift_streaming_analytics.sql -- BLOQUE 1.2
CREATE MATERIALIZED VIEW mv_clicks_stream_raw ... AS
SELECT
    kinesis.approximate_arrival_timestamp,
```

```

kinesis.shard_id,
kinesis.sequence_number,
CAN_JSON_PARSE(FROM_VARBYTE(kinesis.kinesis_data, 'utf-8')) AS es_json_valido,
FROM_VARBYTE(kinesis.kinesis_data, 'utf-8') AS payload_texto,
...
FROM ext_kinesis."clicks-ecommerce" AS kinesis;
-- Los invalidos no se pierden: quedan aislados para inspeccion
CREATE OR REPLACE VIEW v_clicks_cuarentena AS
SELECT approximate_arrival_timestamp, shard_id, sequence_number, payload_texto
FROM mv_clicks_stream_raw WHERE es_json_valido = FALSE;

```

El motivo es la resistencia al *schema drift*. Si mañana el productor cambia la estructura del JSON o emite un registro corrupto, **la ingesta no se rompe**: el registro entra igual, se marca como inválido y queda disponible en `v_clicks_cuarentena` para inspección. Con un casteo directo en la vista materializada, un solo registro malformado abortaría el refresco completo y detendría la ingesta de todo el stream.

12. Análisis de fallos: la saturación de un shard

Requisito literal del enunciado: «Explica qué pasaría si un shard de Kinesis se satura y cómo lo detectaría tu sistema.»

Este análisis no es hipotético: el sistema registró el evento durante la prueba

En el gráfico de alarmas de la Figura 7 y en el panel de monitoreo de la Figura 15 se observa un pico de `ReadProvisionedThroughputExceeded` alrededor de las 23:30 UTC. Es decir: durante la corrida evidenciada, la cuota de lectura de los shards se agotó momentáneamente y Kinesis rechazó llamadas `GetRecords`. Lo que sigue explica por qué ocurrió, qué habría pasado de sostenerse, y cómo el sistema lo detectó.

12.1 Los dos modos de saturación

	Saturación de ESCRITURA	Saturación de LECTURA
Límite del shard	1 MB/s o 1.000 registros/s	2 MB/s compartidos entre todos los consumidores
Quién la provoca	El productor emite más de lo que el shard admite, o la clave de partición concentra el tráfico en un solo shard (<i>hot shard</i>)	Demasiados consumidores leyendo del mismo shard, o lecturas demasiado frecuentes
Qué devuelve Kinesis	<code>ProvisionedThroughputExceededException</code> en <code>PutRecord</code>	<code>ProvisionedThroughputExceededException</code> en <code>GetRecords</code>
Métrica	<code>WriteProvisionedThroughputExceeded</code>	<code>ReadProvisionedThroughputExceeded</code>
Alarma desplegada	<code>kinesis-write-throttled-*</code> OK durante toda la prueba	<code>kinesis-read-throttled-*</code> registró el pico descrito arriba
Consecuencia inmediata	El productor recibe el error. Si no reintenta, el evento se pierde en origen : Kinesis nunca lo recibió.	El consumidor reintenta con <i>backoff</i> . No se pierde nada, pero la lectura se ralentiza y el <code>Iterator Age</code> empieza a crecer.

12.2 Por qué se saturó la lectura en esta prueba

La causa es directamente atribuible a la arquitectura descrita en §2.2: los 2 MB/s de cuota de lectura por shard **se reparten entre todos los consumidores**, y este stream tiene tres leyendo simultáneamente:

Consumidor	Qué hace	Patrón de lectura
Job de PyFlink	clicks-to-iceberg	Polling continuo sobre ambos shards
Redshift Streaming Ingestion	Refresco de mv_clicks_stream_raw	Ráfagas intensas en cada REFRESH
Kinesis Data Firehose	Entrega hacia la zona raw de S3	Polling continuo

El pico coincide temporalmente con los refrescos manuales de la vista materializada, que son precisamente el patrón más agresivo: Redshift intenta ponerse al día leyendo desde TRIM_HORIZON lo más rápido que puede, mientras Flink y Firehose siguen consumiendo su parte.

Por qué el pico no degradó el resultado

El *throttling* de lectura no pierde datos: el consumidor reintenta con backoff exponencial. La evidencia lo confirma desde tres ángulos independientes: el Iterator Age se mantuvo plano (Fig. 6), skipped_rows = 0 en ambos shards (Fig. 11), y la latencia de ingesta se mantuvo en 2–3 segundos (Fig. 9 y 10). El sistema absorbió la saturación sin degradación observable. La alarma volvió a OK por sí sola.

12.3 La cadena de detección, en orden de aparición

La pregunta del enunciado —«cómo lo detectaría tu sistema»— tiene una respuesta ordenada: las señales no aparecen todas a la vez, sino en una secuencia que permite distinguir un pico transitorio de una saturación estructural.

Orden	Señal	Fuente	Qué indica
1	ReadProvisionedThroughputExceeded > 0	CloudWatch · alarma desplegada por Terraform	Hay throttling ahora . Por sí sola no implica degradación: puede ser un pico absorbido por el backoff. Es la señal que se observó.
2	GetRecords.IteratorAgeMillis crecientemente	CloudWatch · alarma a 60 s sostenidos	La señal decisiva. Si el throttling es transitorio, el Iterator Age no se mueve. Si crece de forma sostenida, el consumidor ya no se recupera y la saturación es estructural.
3	GetRecords.Success < 1	CloudWatch · panel de Kinesis	Proporción de llamadas rechazadas. Cuantifica la severidad.
4	Backpressure en el Flink Dashboard	Pestaña <i>BackPressure</i> del job	Si el cuello de botella estuviera aguas abajo (S3 lento, por ejemplo) y no en la lectura, aparecería aquí. Permite distinguir el origen.
5	Lag en sys_stream_scan_states	Redshift · vista de sistema	Confirma si el consumidor de Redshift en particular se está quedando atrás.
6	Frescura del dato más reciente	SQL sobre v_clicks_tipado	El síntoma que percibe el negocio: los tableros muestran datos viejos.

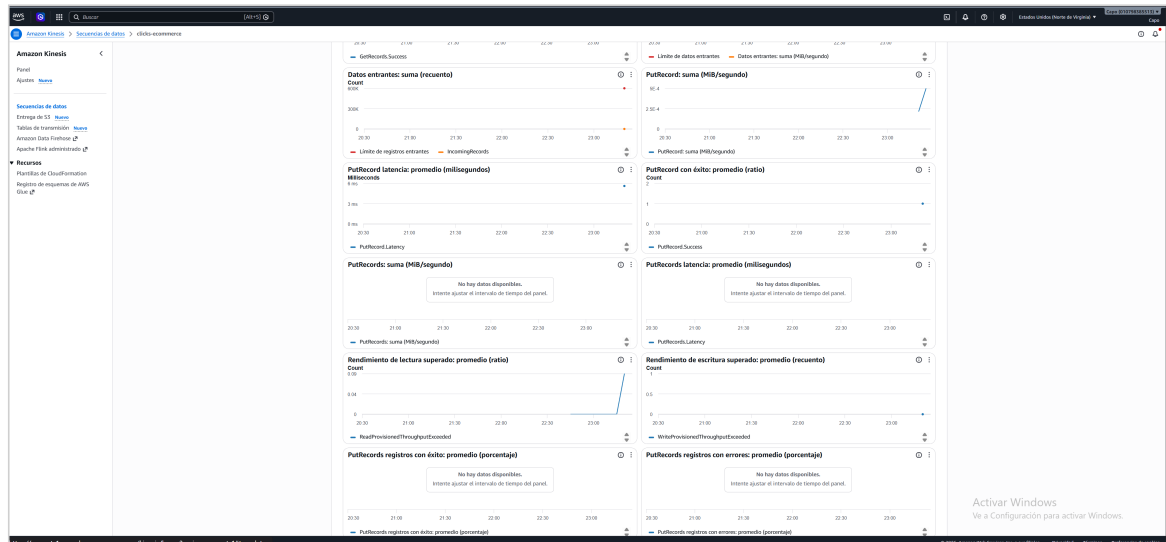


Figura 15. Panel de monitoreo de Kinesis, sección inferior. El gráfico *Rendimiento de lectura superado* (ReadProvisionedThroughputExceeded) registra el pico descrito, mientras *Rendimiento de escritura superado* (WriteProvisionedThroughputExceeded) permanece en cero: la saturación fue exclusivamente de lectura, coherente con tener tres consumidores compartiendo la cuota.

12.4 Qué habría pasado si la saturación se hubiera sostenido

Horizonte	Consecuencia
Segundos	Los consumidores reintentan con backoff. Sin impacto observable. Es lo que ocurrió.
Minutos	El Iterator Age comienza a crecer. La alarma dispara a los 2 minutos sostenidos por encima de 60 s. Los datos calientes de Redshift envejecen.
Horas	El retraso se acumula. Flink sigue procesando correctamente —el <i>event time</i> protege la exactitud de las ventanas— pero cada vez más tarde. Los tableros dejan de ser «tiempo real».
24 horas	Pérdida real de datos. Los registros más antiguos expiran por retención antes de ser leídos. Único punto de la cadena donde la información desaparece de forma irrecuperable.

12.5 Remediación, en orden de intervención

Acción	Cuándo aplica	Efecto y costo
Enhanced Fan-Out	Saturación de lectura con varios consumidores — el caso observado	Da a cada consumidor su propia cuota de 2 MB/s por shard en lugar de compartirla, y reduce la latencia de propagación. Es la solución arquitectónicamente correcta aquí. Cuesta por consumidor-shard-hora más los datos recuperados.
Aumentar shards	Saturación de escritura , o de lectura por volumen real	shard_count es una variable de Terraform: el cambio es una línea y un apply. Requiere subir el parallelism de Flink en consecuencia y activar el lock manager de DynamoDB, ya provisto.
Espaciar los refrescos	Cuando el pico coincide con el REFRESH de Redshift	Reduce la presión de lectura sin costo adicional, a cambio de menor frescura. Es el trade-off de \$9.5.
Revisar la clave de partición	Un shard saturado y el otro ocioso (<i>hot shard</i>)	Añadir shards no resuelve una distribución desbalanceada: el tráfico seguiría cayendo en el mismo. Hay que cambiar la clave.

Acción	Cuándo aplica	Efecto y costo
Modo on-demand	Carga impredecible o muy variable	Kinesis escala los shards solo. Elimina el problema de dimensionamiento a cambio de un costo por GB significativamente mayor.

El diagnóstico correcto importa más que la acción

Las dos primeras filas del cuadro resuelven problemas distintos y confundirlas es caro. En el caso observado —throttling de lectura con tres consumidores y un volumen real ínfimo— **añadir shards habría sido la respuesta equivocada**: multiplicaría el costo fijo del stream sin corregir la causa, que es el reparto de una cuota compartida. La métrica que distingue ambos escenarios es cuál de las dos alarmas de *throttling* disparó, y por eso Terraform despliega las dos por separado en lugar de una sola genérica.

13. Seguridad y principio de menor privilegio

Criterio 5 del enunciado: «Aplicación estricta de políticas de IAM. Ningún componente debe tener permisos * sobre recursos de producción.»

13.1 Inventario de políticas

El entorno despliega 14 políticas IAM. Trece están acotadas a ARN concretos; una sola conserva `Resource = "*"`, por una limitación de la API de AWS que se documenta y se mitiga a continuación.

Rol / política	Permisos	Alcance
flink-execution-dev kinesis-read	DescribeStream, GetRecords, GetShardIterator, ListShards	ARN exacto del stream <code>clicks-ecommerce</code>
flink-execution-dev s3-checkpoints / code-read	Lectura y escritura de objetos	Prefijos concretos del bucket, no el bucket entero
flink-execution-dev cloudwatch-logs	PutLogEvents, CreateLogStream	"\${aws_cloudwatch_log_group.flink_log_group.arn} : *" — el comodín está dentro del ARN del grupo de logs, que es la forma correcta de referirse a sus streams
flink-execution-dev glue-access / iceberg-lock	Operaciones de catálogo y bloqueo	Base <code>lakehouse_db</code> y tabla <code>iceberg_glue_lock</code>
redshift-serverless-dev kinesis-streaming	Lectura del stream y descifrado	ARN del stream. <code>kms:Decrypt</code> acotado a <code>data.aws_kms_alias.kinesis.target_key_arn</code> , no a todas las claves de la cuenta
redshift-serverless-dev glue-lakehouse	Lectura del catálogo y de los datos	Base y prefijo del lakehouse
firehose-kinesis-dev	Entrega hacia S3 y logs	Bucket destino y el grupo de logs gestionado por Terraform
role-data-processing-dev	Acceso a datos	Bucket del data lake
role-control-plane-audit-dev AuditBucketReadOnly	Solo lectura	<code>var.bucket_arn</code>
role-control-plane-audit-dev AuditIamReadOnly	Solo lectura de IAM	Roles de esta cuenta
role-control-plane-audit-dev AuditControlPlaneReadOnly	Solo lectura del plano de control <code>Describe*</code> / <code>List*</code>	Resource = "*" — única del entorno. Ver justificación abajo.

13.2 La única excepción, y su mitigación

Por qué esa política no puede acotarse por ARN

Las acciones de descripción del plano de control de Kinesis y Firehose (`DescribeStreamSummary`, `ListStreams`, `ListDeliveryStreams`) **no admiten un ARN de recurso en la política**: son operaciones a nivel de cuenta y AWS exige `Resource = "*"`. Es una limitación documentada de la API, no una omisión del diseño. La política se construyó para minimizar el alcance dentro de esa restricción.

```
# terraform/environments/dev/modules/identity/main.tf
{
  Sid    = "AuditControlPlaneReadOnly"
  Effect = "Allow"
  Action = [
    # lista explicita: ningun comodin en la accion
    "kinesis:ListStreams",
    "kinesis:DescribeStreamSummary",
    "firehose:ListDeliveryStreams",
    "firehose:DescribeDeliveryStream",
    "glue:GetDatabases",
    "glue:GetTables"
  ]
  Resource = "*"
  # exigido por la API: son acciones de cuenta
  Condition = {
    # ... pero acotado por region
    StringEquals = { "aws:RequestedRegion" = data.aws_region.current.name }
  }
}
```

Tres mitigaciones concurrentes sobre esa política: **(a)** la lista de acciones es explícita y contiene solo verbos de lectura, nunca `kinesis:*`; **(b)** una condición `aws:RequestedRegion` la restringe a `us-east-1`, de modo que el rol no puede enumerar recursos en ninguna otra región; **(c)** el rol es exclusivamente de auditoría y no tiene ningún permiso de escritura en el entorno.

13.3 Seguridad más allá de IAM

Control	Implementación
Cifrado en tránsito y reposo	Kinesis cifrado con KMS. El tráfico hacia S3 no sale a internet: circula por un VPC endpoint de tipo Gateway.
Gestión de credenciales	El namespace de Redshift usa <code>manage_admin_password = true</code> : AWS genera y rota la contraseña en Secrets Manager. Ninguna credencial aparece en el código ni en el estado de Terraform en texto plano.
Exposición de red	El workgroup de Redshift vive en tres subredes privadas, con <code>publicly_accessible = false</code> y su propio grupo de seguridad.
Separación de privilegios en SQL	Rol <code>analytics_reader</code> con <code>SELECT</code> sobre las vistas modeladas. Deliberadamente no se le concede USAGE sobre ext_kinesis : un analista consulta el modelo, nunca la fuente cruda del stream.
Autenticación de usuarios	<code>CREATE USER analyst_user PASSWORD DISABLE</code> fuerza acceso federado por IAM: no existe contraseña que robar ni rotar.

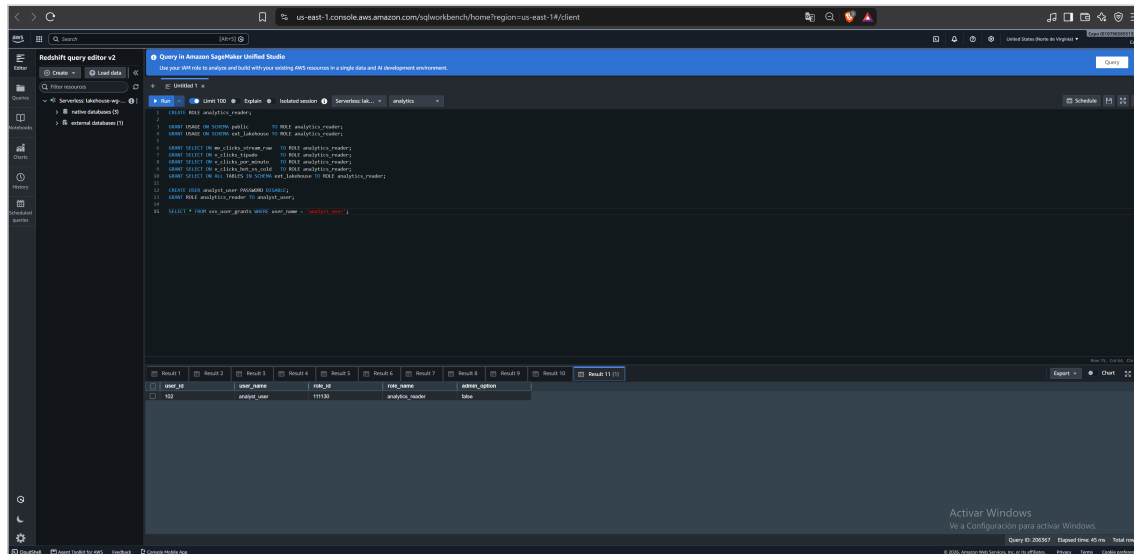


Figura 16. Verificación del RBAC en Redshift: `svv_user_grants` confirma que `analyst_user` tiene concedido el rol `analytics_reader` con `admin_option = false`, es decir, sin capacidad de propagar el privilegio a otros usuarios.

14. Gestión del ciclo de vida con Terraform

Requisito del enunciado: «explicar cómo Terraform gestiona el ciclo de vida de todo el stack». Y del criterio de evaluación: «¿El repositorio usa módulos y backend remoto de forma correcta?»

14.1 Dos ciclos de vida separados a propósito

Capa	Qué contiene	Frecuencia de cambio	Estado
terraform/bootstrap	Bucket de estado versionado y tabla DynamoDB de bloqueo	Se crea una vez y no se toca	Local (no puede guardarse a sí mismo)
environments/dev	Los 45 recursos del pipeline	Se crea y destruye en cada sesión de trabajo	Remoto en el bucket del bootstrap

La separación es lo que permite destruir el entorno completo sin perder el historial de estado, y es también la razón por la que el destroy del final de cada sesión se ejecuta exclusivamente sobre `environments/dev`.

```
# terraform/environments/dev/backend.tf
terraform {
  backend "s3" {
    bucket      = "...-tfstate-..."
    key         = "environments/dev/terraform.tfstate"
    region      = "us-east-1"
    encrypt     = true
    dynamodb_table = "...-tflock"    # bloqueo: impide applies concurrentes
  }
}
```

El bloqueo no es decorativo: aparece en la salida de cada ejecución (`Acquiring state lock. This may take a few moments...`) y es lo que impide que dos operadores apliquen cambios simultáneos y corrompan el estado.

14.2 Modularidad y contratos explícitos

El entorno se compone de seis módulos que se comunican por variables y outputs, nunca por referencias cruzadas a recursos internos. El módulo de Kinesis no sabe qué hace el de Flink con el ARN que expone; solo garantiza exponerlo. Eso hace que cada módulo sea sustituible sin reescribir los demás.

```
# terraform/environments/dev/main.tf (extracto)
module "kinesis" {
  source      = "../modules/kinesis"
  environment = var.environment
  bucket_arn  = aws_s3_bucket.raw_bucket.arn
}

module "flink" {
  source      = "../modules/flink"
  kinesis_stream_arn = module.kinesis.stream_arn      # contrato entre modulos
  code_bucket_name  = aws_s3_bucket.raw_bucket.id
  app_artifact_path = "${path.root}/../../flink-app/clicks_processor.zip"
}
```

14.3 El ciclo completo, comando por comando

Fase	Comando	Qué garantiza Terraform
Inicialización	terraform init	Descarga proveedores y conecta con el backend remoto. Si otro operador tiene el bloqueo, espera.
Planificación	terraform plan	Refresca el estado real contra AWS y calcula el delta. Aborta si no puede leer el estado (§5.2), en lugar de operar sobre una imagen incompleta.
Aplicación	terraform apply	Crea los recursos en orden de dependencia, resuelto a partir del grafo. Verificado: 45 recursos, 0 errores .
Convergencia	terraform apply (repetido)	0 added, 0 changed sobre la infraestructura. Idempotencia demostrada (§5.2).
Evolución	editar .tf + apply	Cambiar el número de shards o las RPU de Redshift es una línea. Terraform decide si actualiza en sitio o recrea.
Destrucción	terraform destroy	Elimina todo en orden inverso de dependencia. Es el control de costo principal del proyecto (§16).

14.4 Controles de costo integrados en el código

El enunciado advierte sobre «ignorar el costo». El módulo de Flink incorpora un interruptor documentado en la propia declaración de la variable, para que quien lo lea vea el costo antes de aplicar:

```
# terraform/environments/dev/modules/flink/variables.tf
variable "enable_managed_flink" {
  type      = bool
  description = <<-EOT
    Despliega la aplicacion de Amazon Managed Service for Apache Flink.
    ATENCION AL COSTO: este recurso factura por KPU-hora mientras EXISTE
    (~USD 0,11/KPU-hora, unos USD 80/mes si se deja corriendo). Para entornos
    de estudio, ponerla en false o destruir el stack al terminar la sesion.
  EOT
  default    = true
}
```

}

El patrón `count = var.enable_managed_flink ? 1 : 0` permite desactivar el componente más caro sin borrar su código ni perder la capacidad de reactivarlo con un solo booleano.

15. Hallazgos de auditoría

Esta sección documenta las limitaciones conocidas del sistema. Se incluye porque un documento de auditoría que solo presenta lo que funcionó no cumple su función: el valor de una auditoría está en lo que encuentra.

15.1 Hallazgo A — La aplicación de Managed Flink no alcanza RUNNING

Severidad	Media — no afecta al pipeline principal
Estado	La aplicación se despliega correctamente con Terraform (criterio 1 cumplido, Fig. 2) y queda en estado READY. No llega a RUNNING.
Causa raíz	El job <code>clicks_processor.py</code> usa la <code>DataStream</code> API con <code>FlinkKinesisConsumer</code> , que requiere el conector de Kinesis en el classpath. Managed Flink no lo trae preinstalado : hay que declararlo en el artefacto.

Diagnóstico realizado —tres intentos documentados:

#	Hipótesis y acción	Resultado
1	El JAR no está en el classpath. Se revisó el código y en efecto <code>env.add_jars(...)</code> estaba comentado.	Python process exits with code: 1 <code>CodeError.InvalidApplicationCode</code>
2	Declarar el JAR con la propiedad <code>jarfile</code> del grupo <code>kinesis.analytics.flink.run.options</code> , empaquetando con <code>Compress-Archive</code> de PowerShell.	We couldn't find the configured file 'lib/flink-sql-connector-kinesis-1.15.4.jar' in your zip file
3	Hallazgo intermedio: <code>Compress-Archive</code> escribe las rutas internas del ZIP con separador de Windows (<i>backslash</i>), violando el estándar ZIP. Se escribió <code>scripts/build_flink_zip.py</code> con <code>zipfile</code> de la biblioteca estándar, que normaliza a <i>«/»</i> , con verificación explícita de que no queden <i>backslashes</i> . Terraform subió los 57,5 MB correctamente.	El mismo error persiste. El ZIP está bien formado y el servicio sigue sin encontrar el archivo.

Decisión tomada: se detuvo la investigación tras el tercer intento y se revirtió la propiedad `jarfile` a un bloque comentado en el módulo, con el hallazgo documentado en el propio código, de modo que el `terraform apply` completa sin errores. La hipótesis pendiente más probable es que Managed Flink espere el contenido bajo un directorio raíz dentro del ZIP y que las rutas de la propiedad deban incluir ese prefijo.

Por qué esta limitación no compromete el proyecto

La aplicación de Managed Flink corresponde al job de conteo a consola de una entrega anterior; **no escribe en Iceberg y no participa del pipeline evidenciado**. El procesamiento en streaming que sostiene el sistema es `flink-app/iceberg_processor.py`, que se ejecuta contra el Kinesis real desplegado por Terraform, escribe en Iceberg, commitea en Glue y está documentado con checkpoints, dashboard sin excepciones y datos consultables desde Athena y Redshift (§6). El criterio 1 del enunciado —que Terraform despliegue la Flink App— se cumple: la aplicación existe, la creó Terraform y la Figura 2 lo evidencia.

15.2 Hallazgo B — Throttling de lectura por cuota compartida

Severidad	Baja — absorbido sin degradación observable
Observación	Pico de <code>ReadProvisionedThroughputExceeded</code> a las ~23:30 UTC, coincidente con los refrescos de la vista materializada.
Causa	Tres consumidores (Flink, Redshift, Firehose) compartiendo los 2 MB/s de cuota de lectura por shard.
Impacto	Ninguno medible: <code>Iterator Age</code> plano, <code>skipped_rows</code> = 0, latencia estable en 2–3 s. La alarma volvió a OK sola.
Remediación	Enhanced Fan-Out, que asigna cuota dedicada por consumidor. No añadir shards: multiplicaría el costo sin atacar la causa (§12.5).

15.3 Hallazgo C (resuelto) — Conflicto de classloaders con Dropwizard Metrics

Se documenta por su valor diagnóstico: es el caso de estudio de por qué existen `tolerable_checkpoint_failure_number` = 0 y el hilo vigilante descritos en §10.

Síntoma: el job escribía archivos Parquet en S3, los checkpoints se reportaban completados, pero el `metadata_location` de Glue permanecía clavado en 000000. La tabla existía, tenía datos debajo, y toda consulta devolvía vacío. Cientos de archivos huérfanos acumulados y ningún mensaje de error.

Diagnóstico: el hilo vigilante que espera `result.wait()` capturó la causa raíz que el minicluster estaba descartando al apagarse:

```
java.lang.LinkageError: loader constraint violation: loader ... previously
initiated loading for a different type with name
"com/codahale/metrics/Histogram"
    at org.apache.iceberg.flink.sink.IcebergStreamWriter.flush(...)
```

Causa raíz: `iceberg-flink-runtime` trae *bundleado* `com.codahale.metrics`, y PyFlink ya incluye `flink-metrics-dropwizard` en su classpath. La misma clase cargada por dos classloaders distintos hace explotar `IcebergStreamWriter.flush()` en el primer checkpoint que tenga datos reales que escribir — no en los vacíos, lo que explica que los primeros checkpoints parecieran exitosos.

```
# flink-app/iceberg_processor.py -- la correccion, dos lineas
conf.setString(
    "classloader.parent-first-patterns.additional",
    "com.codahale.metrics;org.apache.flink.dropwizard",
)
```

Forzar *parent-first* para esos paquetes hace que ambos usen la misma clase. Tras el cambio, el `metadata_location` empezó a avanzar y el pipeline quedó operativo. **Es el hallazgo que convirtió este**

proyecto en un sistema funcional, y solo fue diagnosticable porque el job estaba configurado para fallar ruidosamente.

15.4 Deuda técnica reconocida

Ítem	Situación actual	Qué correspondería en producción
Estrategia de reinicio	<code>restart-strategy = none</code>	<code>exponential-delay</code> con tope, y alarma sobre <code>fullRestarts</code>
Paralelismo	1 escritor	Paralelismo igual al número de shards, con el lock manager de DynamoDB activado (ya provisto por Terraform)
Archivos huérfanos	Sin limpieza programada	<code>remove_orphan_files</code> y compactación de archivos pequeños como tarea periódica
Conector de Kinesis para Managed Flink	Hallazgo A abierto	Resolver el empaquetado del artefacto, o migrar el job a la Table API con conector declarado en SQL
Alerta de frescura en Redshift	Se consulta manualmente	Métrica personalizada de CloudWatch sobre <code>MAX(approximate_arrival_timestamp)</code> con su alarma
Enhanced Fan-Out	Cuota de lectura compartida	Activarlo si se suman consumidores o crece el volumen

15.5 Mejoras versionadas con posterioridad a la corrida evidenciada

Estos recursos están en el repositorio pero NO formaban parte del despliegue de 45 recursos

Las tres adiciones que siguen se escribieron después de capturar la evidencia de §6 y responden a huecos detectados en la propia auditoría. Están versionadas y comentadas en el repositorio, pero **no se aplicaron en la corrida documentada**, de modo que las cifras de §5 y §6 no las incluyen. Se declaran aquí para que el recuento de recursos del código y el del documento puedan reconciliarse.

Recurso	Qué resuelve	Archivo
Planificador de refresco EventBridge Scheduler + Redshift Data API, rate(1 minute)	§9.5 define la <i>estrategia</i> de refresco de 60 s, pero el mecanismo quedaba en <code>AUTO REFRESH</code> , que es best-effort: Redshift decide cuándo refrescar según la carga. Con el planificador el intervalo es determinístico y auditable.	<code>modules/redshift/main.tf</code> §5
Alarmas de lag por fragmento métricas de shard + una alarma por <code>ShardId</code>	Las alarmas de §6.4 son agregadas por stream. Un único shard atrasado —un <i>hot shard</i> por clave de partición mal distribuida— queda enmascarado por el promedio. Sin desglose por <code>ShardId</code> no se puede distinguir ese caso de una saturación general, y la remediación es opuesta (§12.5).	<code>modules/kinesis/main.tf</code>
Rol IAM del analista credenciales temporales acotadas al workgroup	<code>CREATE USER . . . PASSWORD DISABLE</code> (§13.3) obliga a autenticación federada, pero no había ninguna vía declarada para obtenerla. Este rol es la contraparte: solo <code>GetCredentials</code> sobre este workgroup, con denegación explícita del secreto del administrador.	<code>modules/redshift/main.tf</code> §6

El script `sql/redshift_streaming_analytics.sql` incorpora, en la misma línea de trabajo, tres cambios que tampoco estaban en la corrida evidenciada: el parseo tipado de cinco campos JSON dentro de la propia vista materializada —con guarda de validez para conservar la resistencia al *schema drift* de §11.4—, un bloque de verificaciones previas a la consulta federada, y una prueba ejecutable de permisos

denegados para el rol analítico.

16. Síntesis de trade-offs

Recapitulación de las decisiones de diseño del sistema, con la alternativa descartada y el criterio que resolvió cada una.

Decisión	Alternativa descartada	Por qué
Dos consumidores en paralelo sobre Kinesis	Encadenar Kinesis → Flink → Iceberg → Redshift	Encadenar impondría a la analítica en vivo la latencia completa del camino de batch. El costo es compartir la cuota de lectura, que es exactamente el hallazgo B.
Checkpoint de 30 s	5 s / 5 min	Más corto: archivos pequeños y sobrecarga de snapshots. Más largo: ventana de datos no publicados y más trabajo a rehacer tras un fallo.
Watermark de 10 s	1 s / 5 min	Triplifica con holgura la latencia real medida (2–3 s) sin retrasar el cierre de ventanas de forma perceptible.
Parallelism 1	Paralelismo = nº de shards	Evita contención de commits en Glue y hace innecesario el lock manager. Suficiente para 2 shards y este volumen.
Tolerancia cero a checkpoints fallidos	El valor por defecto, que tolera fallos	Un checkpoint fallido significa que Iceberg no commiteó. Es preferible un job muerto y visible a un lakehouse desactualizado en silencio.
Iceberg sobre Glue	Parquet plano con crawler	Evolución de esquema, viaje en el tiempo por snapshot, commits atómicos y el mismo catálogo para Athena y Redshift Spectrum.
Particionar por product_id	Particionar por fecha	Es la clave de filtro dominante de las consultas analíticas. Con muchos productos convendría revisar para no generar particiones minúsculas.
MV sin casteo + cuarentena	Castear en la vista materializada	Un registro malformado abortaría el refresco y detendría la ingesta de todo el stream.
Redshift Serverless	Redshift provisionado	No factura en reposo y escala solo. Para carga sostenida y predecible, un clúster reservado sale más barato.
Refresco manual en la demo	AUTO REFRESH YES	Control del instante del refresco para poder medirlo. En operación continua se activa el automático.
Firehose hacia la zona raw	Solo Flink	Red de seguridad: conserva los eventos originales más allá de las 24 h del stream, por si hubiera que reprocesar con otra lógica.

17. Control de costos

Error común señalado por el enunciado: «Asegúrate de que las ventanas de Flink y la retención en Kinesis no generen costos innecesarios por falta de limpieza o dimensionamiento excesivo.»

Componente	Modelo de cobro	Decisión adoptada
Kinesis Data Streams	Por shard-hora + PUT payload units	2 shards, el mínimo con paralelismo real. Retención en 24 h, el mínimo del servicio.
Managed Flink	Por KPU-hora, mientras el recurso exista	El componente más caro (~USD 80/mes si se deja corriendo). Interruptor enable_managed_flink documentado, y destroy al terminar cada sesión.

Componente	Modelo de cobro	Decisión adoptada
Redshift Serverless	Por RPU-hora consumida	8 RPU (el mínimo). No factura en reposo. El refresco manual evita consumir RPU refrescando una vista que nadie consulta.
S3 + Glue	Por GB almacenado y peticiones	<code>write.metadata.delete-after-commit.enabled</code> con 20 versiones. Sin esto, un commit cada 30 s deja miles de <code>metadata.json</code> en pocos días.
Athena	Por bytes escaneados	Particionado por <code>product_id</code> + compresión <code>zstd</code> . Cada byte no escaneado es un byte no facturado.
CloudWatch	Por alarma y por métrica	3 alarmas, las mínimas para cubrir los dos modos de saturación y el lag del consumidor.

El control de costo principal es operativo, no de configuración

Kinesis factura por shard-hora y Managed Flink por KPU-hora **mientras los recursos existan**, se usen o no. Ninguna optimización de parámetros compensa dejar el entorno encendido. Por eso el procedimiento de trabajo termina siempre con `terraform destroy` sobre `environments/dev`, y por eso la separación del bootstrap (§14.1) es lo que hace que destruir sea barato y reversible en lugar de traumático.

18. Reproducción por un auditor externo

Error común señalado por el enunciado: «entregar código que "funciona en mi máquina" pero que no tiene instrucciones claras para un auditor externo». Esta sección es el procedimiento completo, sin pasos implícitos.

18.1 Requisitos previos

- Credenciales de AWS con permisos de administrador sobre una cuenta de pruebas.
- Terraform ≥ 1.5 , Python 3.11 y Java 11 en la máquina del auditor.
- `pip install apache-flink==1.18.1 boto3`
- Los JAR de los conectores en `flink-app/lib/` y `flink-app/lib_iceberg/`. Están excluidos del control de versiones por tamaño; el README documenta las URL de descarga desde Maven Central.

18.2 Procedimiento

```
# 1. Backend remoto (una sola vez por cuenta)
cd terraform/bootstrap && terraform init && terraform apply

# 2. Entorno completo: los 45 recursos, sin pasos manuales
cd ../environments/dev && terraform init && terraform apply -auto-approve

# 3. El unico acoplamiento entre infraestructura y aplicacion: un output
$env:LAKEHOUSE_BUCKET = terraform output -raw raw_bucket_name

# 4. Ejecutar el pipeline. El script lanza el productor por su cuenta.
cd ../../..
python -u flink-app/iceberg_processor.py
# -> imprime el JobID y el estado de los checkpoints cada 20 s
# -> Flink Dashboard en http://localhost:8081
# -> NO cortar con Ctrl+C: el job se cancela solo al terminar

# 5. Verificar que Iceberg commiteo (metadata debe avanzar de 00000)
./scripts/verify_iceberg.ps1

# 6. Redshift: ejecutar en Query Editor v2, en orden
```

```
# sql/redshift_streaming_analytics.sql bloques 0 a 7, en el orden numerado del encabezado
# 7. IMPRESCINDIBLE: liberar los recursos que facturan por hora
cd terraform/environments/dev && terraform destroy -auto-approve
```

Tres advertencias operativas que ahorran tiempo

(a) El paso 3 no es opcional: el job aborta con un mensaje explícito si LAKEHOUSE_BUCKET no está definida, en lugar de escribir en un bucket equivocado. **(b)** No interrumpir el job con Ctrl+C: eso mata la JVM a través de py4j y puede dejar el último checkpoint a medias; el script se cancela limpiamente solo. **(c)** El paso 7 se ejecuta sobre environments/dev y **nunca** sobre bootstrap, donde vive el estado remoto.

19. Conclusión

El sistema auditado cumple los cinco criterios globales del enunciado, con la salvedad documentada en §15.1:

Criterio	Estado	Evidencia
1. Integridad de infraestructura Terraform despliega el entorno completo sin intervención manual	Cumplido	45 recursos en un solo apply; convergencia a 0 cambios en el segundo. Networking, IAM, Kinesis, Flink App, Glue y Redshift. §5
2. Consistencia de datos Misma semántica en Iceberg y en Redshift	Cumplido	Consulta federada uniendo ambos; Athena y Redshift Spectrum coinciden en el mismo total sobre el mismo snapshot. §7
3. Manejo de temporalidad Event time con watermarks configurados	Cumplido	Watermark de 10 s dimensionado sobre la latencia real medida; idle-timeout para el shard inactivo. §8
4. Observabilidad Iterator Age, checkpoints y freshness	Cumplido	14/14 checkpoints, Iterator Age plano, 3 alarmas en OK, lag de 2–3 s. §6
5. Seguridad IAM estricta, sin permisos *	Cumplido con salvedad	13 de 14 políticas acotadas por ARN. La única con Resource="*" corresponde a acciones de nivel de cuenta que la API no permite acotar, y está restringida por región y limitada a lectura. §13

19.1 Lo que el proyecto demuestra

Más allá del cumplimiento formal, el trabajo deja tres resultados que conviene explicitar. **Primero**, que el pipeline es genuinamente end-to-end: un evento entra por Kinesis y es visible tanto en una vista materializada de Redshift —en 2 a 3 segundos— como en una tabla Iceberg del lakehouse, manteniendo la integridad en todo el recorrido. **Segundo**, que la infraestructura es reproducible: un destroy total seguido de un apply reconstruye el entorno completo sin pasos manuales y sin un solo identificador escrito a mano. **Tercero**, que el sistema es observable: cuando la cuota de lectura de Kinesis se saturó durante la prueba, el sistema lo registró, la evidencia quedó capturada y el análisis de §12 pudo escribirse sobre un hecho medido en lugar de sobre un supuesto.

19.2 Sobre las limitaciones documentadas

Los hallazgos de §15 forman parte deliberada de la entrega. La aplicación de Managed Flink que no arranca está diagnosticada hasta el nivel de causa probable, con tres intentos registrados y la hipótesis pendiente explicitada, y su propiedad conflictiva quedó comentada en el código para que el despliegue no falle. El error de classloader —que durante días se manifestó como una tabla que nunca se actualizaba, sin un solo mensaje de error— se resolvió con dos líneas de configuración, pero solo fue diagnosticable porque el job estaba configurado para fallar de forma ruidosa. Esa es, probablemente, la lección más transferible del proyecto: en un sistema de streaming, **un fallo visible vale más que una ejecución aparentemente sana**.

Repositorio completo: <https://github.com/AndresCapoPlaza/dataops-terraform-aws>

Código de infraestructura (Terraform), aplicación de procesamiento (PyFlink), scripts SQL de Iceberg y Redshift, y las 23 evidencias citadas en este documento.

Anexo. Índice de evidencias

Las 23 capturas de esta corrida están versionadas en el repositorio bajo docs/evidencias/, numeradas en el orden cronológico del pipeline. Las que se reproducen en el cuerpo del documento indican su número de figura.

Archivo	Qué documenta	Fig.
00_terminal_job_inicio.png	Arranque del job: JobID, intervalo de checkpoint, lanzamiento del productor	—
01_flink_dashboard_running.png	Job en RUNNING con 3 tareas; 0 finished / 0 canceled / 0 failed	3
02_flink_checkpoints_completados.png	Checkpoints: 11 disparados, 11 completados, 0 fallidos; los 3 operadores al 100%	4
03_flink_sin_excepciones.png	Pestaña Exceptions: No Root Exception tras 6 min 14 s	5
04_kinesis_iterator_age.png	GetRecords.IteratorAgeMilliseconds plano durante la corrida	6
05_kinesis_stream_2_shards.png	Stream activo, modo aprovisionado, 2 particiones, retención 1 día	—
06_kinesis_incoming_records.png	Registros entrantes y latencia de GetRecords	—
07_kinesis_read_throughput_exceeded.png	Pico de ReadProvisionedThroughputExceeded; escritura en cero	15
08_cloudwatch_alarmas_ok.png	Las 3 alarmas desplegadas por Terraform, todas en estado OK	7
09_redshift_error_secreto_obsoleto.png	Fallo de conexión por secreto del namespace anterior (ver nota al pie)	—
10_redshift_external_schema_kinesis.png	CREATE EXTERNAL SCHEMA ... FROM KINESIS ejecutado	—
11_redshift_mv_streaming_creada.png	Vista materializada de streaming ingestion creada	—
12_redshift_eventos_ingeridos_1078.png	1.078 eventos consultables tras el primer refresco	—
13_producer_trafico_kinesis.png	Productor emitiendo 5 ev/s hacia clicks-commerce	—
14_redshift_consulta_federada_hot_cold.png	Consulta federada: stream + Iceberg en una sola sentencia SQL	14
15_redshift_rbac_analytics_reader.png	analyst_user con rol analytics_reader, sin admin_option	16
16_redshift_lag_ingesta.png	Lag de ingesta: mín. 2 s · prom. 2 s · máx. 3 s sobre 1.078 eventos	10
17_redshift_scan_states_error_columna.png	Error de catálogo en sys_stream_scan_states (ver nota al pie)	—
18_glue_tabla_iceberg.png	Glue: formato Apache Iceberg, Version 5 (Current version)	8
19_managed_flink_app_desplegada.png	Aplicación de Managed Flink creada por Terraform, runtime 1.15	2
20_redshift_scan_states_por_shard.png	Escaneo por shard con skipped_rows = 0 en ambos	11
21_redshift_historial_refresh.png	sys_mv_refresh_history: Refresh successfully updated en 2 s	13
22_redshift_refresh_incremental_1809.png	Segundo refresco: 1.078 → 1.809 eventos	12
23_redshift_arrival_timestamp.png	approximate_arrival_timestamp con valores actualizados y latencia por evento	9

Nota sobre las evidencias 09 y 17

Se conservan deliberadamente dos capturas de error. La 09 corresponde al intento de conectar el Query Editor v2 con el secreto del namespace anterior: al recrearse el entorno, Secrets Manager generó un secreto nuevo y la conexión guardada quedó apuntando a uno inexistente. Es un efecto colateral esperable de destruir y recrear un namespace gestionado, y vale la pena documentarlo para quien reproduzca el procedimiento. La 17 corresponde a un error de catálogo en `sys_stream_scan_states`: la columna `shard_id` no existe en la versión de Redshift utilizada —el identificador de fragmento viaja en `partition_id`—, lo que motivó consultar la vista con `SELECT *`. Ambas correcciones quedaron documentadas como comentarios en `sql/redshift_streaming_analytics.sql`, junto con la que exige calificar todas las relaciones con su esquema al crear una vista `WITH NO SCHEMA BINDING`.

Nota sobre las marcas de tiempo

Las capturas de las consolas de AWS y de los paneles de CloudWatch están en UTC. Las capturas de terminal y del Flink Dashboard local muestran la hora de la máquina del operador (UTC−3). El job arrancó a las 20:21:30 locales, equivalentes a las 23:21:30 UTC; ambas referencias describen el mismo instante.